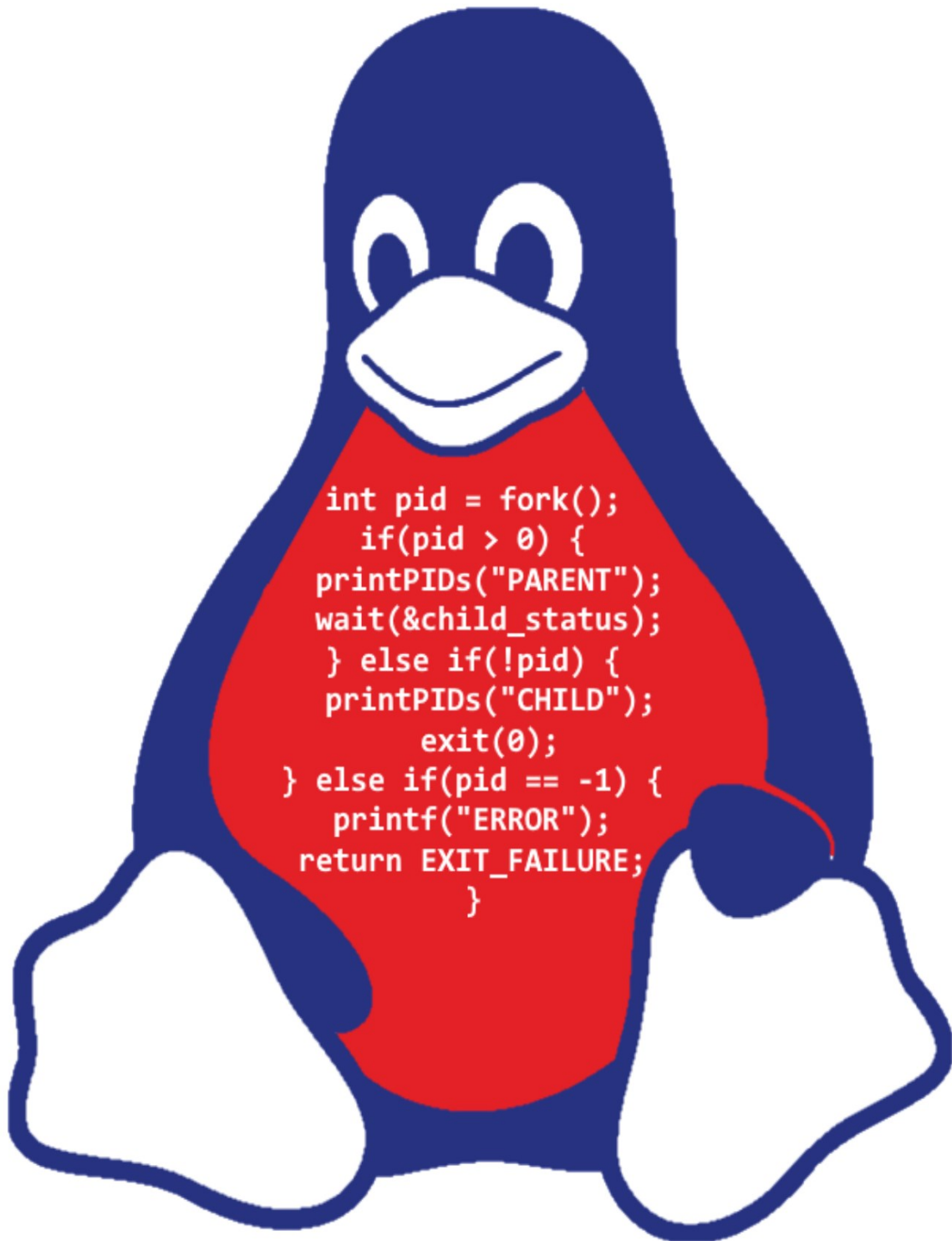
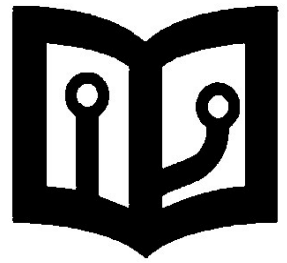




```
int pid = fork();  
    if(pid > 0) {  
        printPIDs("PARENT");  
        wait(&child_status);  
    } else if(!pid) {  
        printPIDs("CHILD");  
        exit(0);  
    } else if(pid == -1) {  
        printf("ERROR");  
        return EXIT_FAILURE;  
    }
```

LINUX SYSTEM PROGRAMMING



+IBA= 

Table of Contents

Introduction	1.1
Chapter 1 - Linux Basics	1.2
What is system programming?	1.2.1
Working Environment	1.2.2
Remote Access	1.2.3
How to get help in Linux?	1.2.4
Files, directories, and file systems	1.2.5
File Transfer Protocol	1.2.6
Processes in Linux	1.2.7
Users and Groups	1.2.8
File Permissions	1.2.9
Manage Users and Groups	1.2.10
Standart Streams	1.2.11
Redirecting and Pipelines	1.2.12
Nano Text Editor	1.2.13
Linux Basics Exercise	1.2.14
Chapter 2 – Programming	1.3
Compiling	1.3.1
Compiler	1.3.2
Source Code	1.3.3
Assembly	1.3.4
Object	1.3.5
Binary	1.3.6
Execute	1.3.7
Libraries	1.3.8
Archiver	1.3.9
Object Files	1.3.10
Static and Dynamic Library	1.3.11
Library Example	1.3.12
Debugging	1.3.13
Programmng Exercise	1.3.14
Chapter 3 – Files and File System	1.4
Files	1.4.1
File System	1.4.2
Wirking with Files	1.4.3
Streams	1.4.4

Files Library	1.4.5
Opening and Closing File	1.4.6
Read File Introduction	1.4.7
Read File Example	1.4.8
Read and Write File Intro	1.4.9
Write File Example	1.4.10
Searching and Sparse Files	1.4.11
Make a sparse file	1.4.12
Lock File Introduction	1.4.13
Lock and Write Example	1.4.14
Files and File System Exercise	1.4.15
Chapter 4 – Processes	1.5
Processes	1.5.1
Processes Management	1.5.2
Processes Methods	1.5.3
Execute Command	1.5.4
Show Source Example	1.5.5
Forking	1.5.6
Print Processes Identifiers	1.5.7
Wait and exit child process	1.5.8
Execute and print state	1.5.9
Zombies and Signals	1.5.10
Zombie Game	1.5.11
Processes Exercise	1.5.12
Chapter 5 – Inter Process Communication	1.6
Inter Process Communication Methods	1.6.1
Signals	1.6.2
Signals List	1.6.3
Signals Management	1.6.4
Signal Handler	1.6.5
Signal Wait	1.6.6
Reviewing Signals	1.6.7
Displaying information about the signals	1.6.8
Reentrancy	1.6.9
Anonymous and Named Pipes	1.6.10
Working with Pipes	1.6.11
Anonymous Pipe Example	1.6.12
Names Pipe Example	1.6.13

Inter Process Communication Exercise	1.6.14
Chapter 6 - Inter Process Communication Synchronization	1.7
Message Queue	1.7.1
Message Queue Libraries	1.7.2
Determining the common data	1.7.3
Message Server Example	1.7.4
Message Client Example	1.7.5
Shared Memory	1.7.6
Shared Memory Libraries	1.7.7
Determining the common data	1.7.8
Shared Memory Server Example	1.7.9
Shared Memory Client Example	1.7.10
Semaphores	1.7.11
Semaphores Libraries	1.7.12
Determining the common data	1.7.13
Semaphore Server Example	1.7.14
Semaphore Client Example	1.7.15
Inter Process Communication Synchronization Exercise	1.7.16
Chapter 7 – Sockets	1.8
Linux Sockets	1.8.1
Working with Sockets in C	1.8.2
Using Unix Sockets	1.8.3
File Socket Server Example	1.8.4
File Socket Client Example	1.8.5
Socket Pair	1.8.6
Socket Pair Example	1.8.7
Network Sockets	1.8.8
Network Socket Server Example	1.8.9
Network Socket Client Example	1.8.10
Sockets Exercise	1.8.11
Chapter 8 – Threads	1.9
Threads Introduction	1.9.1
Threads Library	1.9.2
Threads Management Functions	1.9.3
Creating Threads	1.9.4
Finalizing Threads	1.9.5
Joining Threads	1.9.6
Joining Threads Example	1.9.7

Threads Cancellations	1.9.8
Threads Cancellation Types	1.9.9
Thread Cancellation Example	1.9.10
Threads Exercise	1.9.11
Chapter 9 - Threads Synchronization	1.10
Threads Exit	1.10.1
Managing Threads Exit	1.10.2
Tread Exit Example	1.10.3
Why is synchronization necessary??	1.10.4
Sync Mechanisms	1.10.5
Mutexes	1.10.6
When is Locking Necessary?	1.10.7
Use of Mutexes	1.10.8
Race Conditions	1.10.9
Thhread Safe Code	1.10.10
Deadlock	1.10.11
Creating and Destroying Mutexes	1.10.12
Lock and Unlock Mutexes	1.10.13
Example of synchronization using a mutex	1.10.14
Semaphores Synchronizing	1.10.15
Example of Synchronization Using a Semaphore	1.10.16
Threads Synchronization Example	1.10.17
Chapter 10 – Daemons	1.11
What are the daemons?	1.11.1
Daemon Sceleton	1.11.2
Chat Daemon	1.11.3

Linux System Programming

The current project **Linux System Programming** is a joint initiative of Burgas Free University and IBA Bulgaria, as an additional resource to the free course **Linux System Programming**. The textbook is available for free reading in the form of [electronic book](#). Learning resources are available for free download from [electronic repository](#). All materials are distributed under license [CC-BY-NC-SA](#).

Format	ISBN
PDF	978-619-7126-69-3
MOBI	978-619-7126-70-9
EPUB	978-619-7126-71-6

Author

Dimitar Minchev is a associated professor at the Faculty for Informatics and Technical Sciences at Burgas Free University. Prepares students for the [Republican Student Olympiad in Programming](#) in the [Competitive Programming Club](#). Organizes a competition for the development of desktop and mobile applications [HACKATHON @ BFU](#). He created the unique [Academy for Talents in Programming](#) and [School of Robotics](#) for students from Burgas. Initiates the student programming competition [CODE@BURGAS](#). Participates in the national program [IT Skills and Career Training](#) of the Bulgarian Ministry of Education and Science.

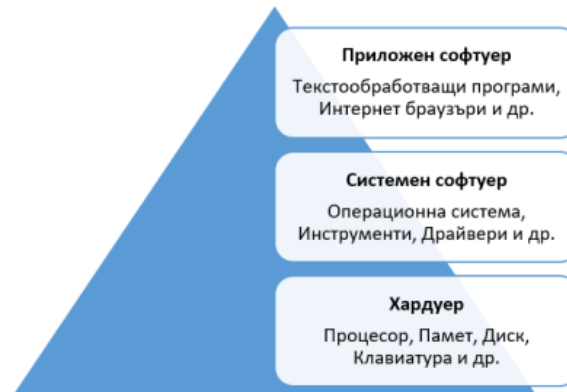
Contact
Work: тел. +359 56 900 477 и e-mail: mitko@bfu.bg
Personal: моб. +359 899 148 872 и e-mail: dimitar.minchev@gmail.com
Blog: http://www.minchev.eu

Chapter 1 - Linux Basics

- [What is system programming?](#)
- [Working Environment](#)
- [Remote Access](#)
- [How to get help in Linux?](#)
- [Files, directories, and file systems](#)
- [File Transfer Protocol](#)
- [Processes in Linux](#)
- [Users and Groups](#)
- [File Permissions](#)
- [Manage Users and Groups](#)
- [Standart Streams](#)
- [Redirecting and Pipelines](#)
- [Nano Text Editor](#)
- [Linux Basics Exercise](#)

What is system programming?

System programming is a discipline that deals with the development of **system software**. The characteristic of system software is that it usually interacts directly with the hardware and system libraries of the **kernel** of the operating system and serves as a platform for other applications, which in most cases are used by the end user or so-called **applied software**.



Operating systems can also be regarded as system software, as they essentially manage all other programs and resources. Other examples include **system drivers, shell, text editor, compiler, debugger, tools** and **daemons**.

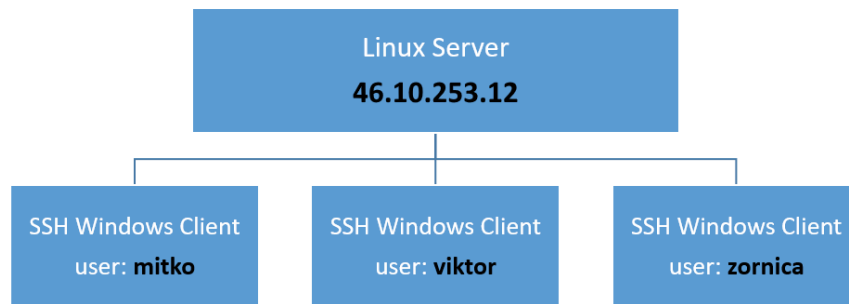
Sometimes a software product can be viewed as both application and system software. Database systems perform system functions for other programs but also offer tools used in the daily work of system operators and data analysts.

In Linux, **system tools** are typically located in the directories **/sbin** and **/usr/sbin**.

System programmers are required to have a good understanding of hardware and operating systems in order to use the system's resources correctly and optimally.

Working Environment

The work environment consists of a Linux server with a real Internet Protocol address **46.10.253.12** and multiple Windows clients. Client access to the server is realized via [Telnet](#) using [SSH](#).



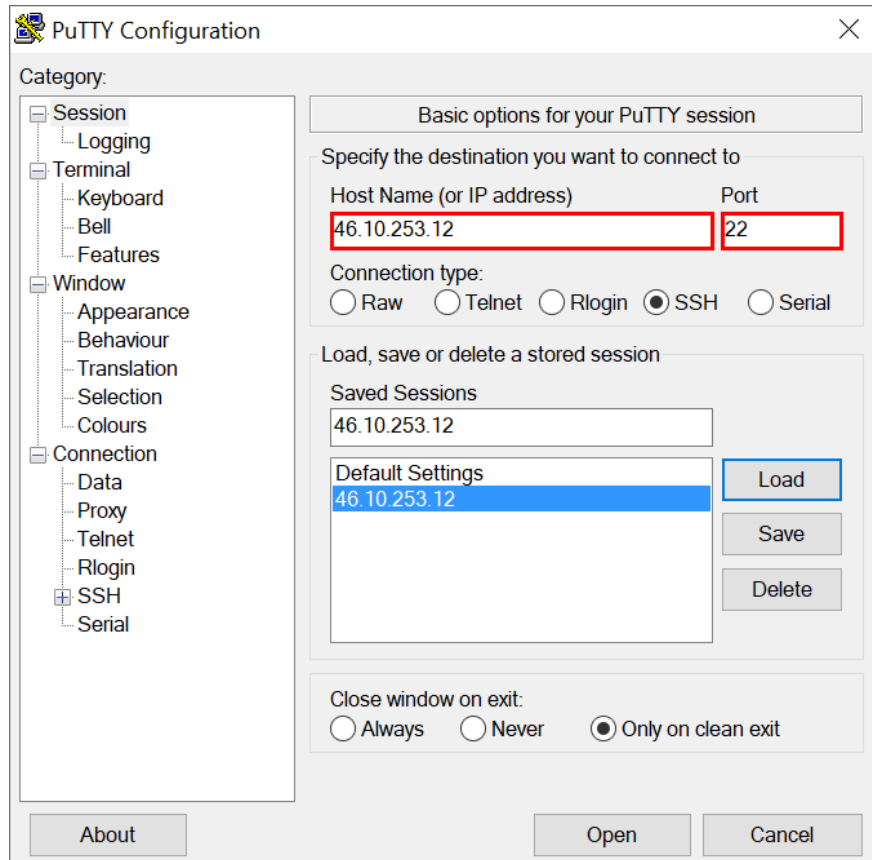
Telnet was developed in 1969, starting with [RFC 15](#), expanded in [RFC 854](#), and standardized as an internet standard [STD 8](#) by the [Internet Engineering Task Force](#), one of the first internet standards. Telnet provides access to a command-line interface on a remote host to networking devices and/or operating systems. Due to serious security concerns when using Telnet over an open network like the Internet, its use for this purpose has significantly decreased in favor of SSH.

SSH uses public key cryptography to authenticate the remote computer and to allow it to authenticate the user if necessary. The protocol is commonly used to log into a remote machine and execute commands. It can transfer files using related file transfer protocols (SFTP) or secure copies (SCP).

The system software of the operating system includes: **shell**, **text editor**, **compiler**, **debugger**, **tools** and **daemons**.

Remote Access

To establish remote access to the server, the program **PuTTY** is used, which is distributed as open-source software and can be downloaded for free from the Internet at the following address: <https://putty.org/>



How to get help in Linux?

Most system applications and commands that run in console mode in the Linux operating system have accompanying documentation in the form of **manual pages**. To access them, you can use the **man** program, with the following format:

```
man <command>
```

where is the name of the application for which you want to get information. If a manual exists for the selected command, man will print the first page on the screen and provide the user with means to navigate the rest. You can see a detailed description of the functions with the following command:

```
man man
```

```
MAN (1)                                Manual pager utils                                MAN (1)
NAME
  man - an interface to the on-line reference manuals

SYNOPSIS
  man [-C file] [-d] [-D] [--warnings[=warnings]] [-R encoding] [-L locale] [-m system[,...]] [-M
  path] [-S list] [-e extension] [-i|-I] [--regex|--wildcard] [--names-only] [-a] [-u] [--no-sub
  pages] [-P pager] [-r prompt] [-7] [-E encoding] [--no-hyphenation] [--no-justification] [-p
  string] [-t] [-T(device)] [-H(browser)] [-X(dpi)] [-Z] [[section] page[.section] ...] ...
  man -k [apropos options] regexp ...
  man -K [-w|-W] [-S list] [-i|-I] [--regex] [section] term ...
  man -f [whatis options] page ...
  man -l [-C file] [-d] [-D] [--warnings[=warnings]] [-R encoding] [-L locale] [-P pager] [-r
  prompt] [-7] [-E encoding] [-p string] [-t] [-T(device)] [-H(browser)] [-X(dpi)] [-Z] file ...
  man -w|-W [-C file] [-d] [-D] page ...
  man -c [-C file] [-d] [-D] page ...
  man [-?V]

DESCRIPTION
  man is the system's manual pager. Each page argument given to man is normally the name of a
  program, utility or function. The manual page associated with each of these arguments is then
  found and displayed. A section, if provided, will direct man to look only in that section of
  Manual page man(1) line 1 (press h for help or q to quit)
```

The important keys that you can use while working in the program are **PgUp** for scrolling up a page, **PgDn** for scrolling down a page, and the **q** key for exit.

Files, directories, and file systems

Files are a fundamental concept in all popular operating systems. This is also completely true for the Linux OS, which is characterized by the idea that “everything is a file.” Files are grouped into **directories**, which in turn can contain other subdirectories, thus creating a tree structure.

The **file system** is the method for structuring and organizing data on the computer. It represents a management table stored on the disk, which the operating system uses to access the files. The entries in this table are called **inode** and can briefly be defined as the serial number of the file.

Basic commands for working with the file system under Linux are presented in the table below:

Command	Description
pwd	Prints the name of the current directory
ls	Lists the contents of the directory
mkdir	Creates a directory
cd	Changes the working directory
rmdir	Deletes a directory
cp	Copies files and directories
mv	Moves and/or renames files
rm	Deletes files and/or directories
ln	Creates a link between files
cat	Concatenates files and prints to standard output

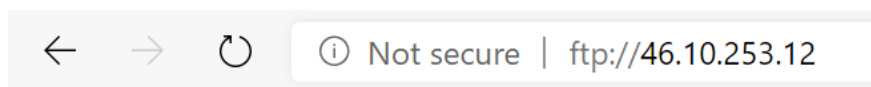
File Transfer Protocol

The operating system provides various tools for transferring information between computers. Typically, the 'client-server' model is used, where clients initiate connections and requests to a server, which responds to them.

We use the [File Transfer Protocol](#). It is a standard network protocol for transferring files between a client and a server in a computer network. You can copy files locally to your folder:

```
student@ubuntu:~$ pwd
/home/student
student@ubuntu:~$ ls
ftp
student@ubuntu:~$ ls ftp/
files
student@ubuntu:~$ █
```

You can access these files with any Internet browser by visiting the address of the work server and identifying yourself with a username and access password:



Index of /

Name	Size	Date modified
 LSP.pdf	2.7 MB	3/16/20, 10:28:00 AM

You can manage the files from your folder with any client that supports the FTP protocol. (For example: [FileZilla](#)).

Processes in Linux

The main task of operating systems is to execute computer programs. When each program is started, one or more **processes** are created, which effectively represent program code in execution. But in addition to static code, they also consist of data, resources, state, and a virtualized computer.

In Linux, each process is represented by a unique identifier called **pid**. When there are no active processes, the operating system idly runs a special **idle process** (Term in English: Idle Process), which has the unique identifier **pid = 0**.

Processes can create other processes, with the original process referred to as the **parent** and the newly created one called the **child**. In this way, hierarchical groups of processes are created, which can communicate and share resources with each other.

Every process is owned by a user and a group registered in the operating system. In most cases, it is the user who launched the application and the corresponding group to which they belong.

Users and Groups

To access the features of Linux, every user must go through an identification process called **login**, where a username and password are entered. After entering the correct combination, the user registers in the operating system and is associated with a unique identifier called **uid**, which represents a positive integer. Since names must also be unique, they are more commonly used in practice than **uid** for user identification.

Each user belongs to one or more user **groups**. This way, the same attributes or permissions can be assigned to multiple users at once, depending on their role in the system.

Authorization for access to resources in Linux is provided through this concept of users and groups. For example, the operations you can perform with files in the OS depend on the permissions that your user account and the group it belongs to possess.

File Permissions

Traditional file systems support three **modes** for file usage: **read**, **write** and **execute**.

On the other hand, every file in Linux belongs to a **user** and a **group**, which by default, are the creator of the file and their primary group. Based on this, three **levels** of access rights for file operations are defined: **owner**, **group** and **public**.

By combining the three levels and the three modes, 9 possible characteristics of files are obtained, which can be presented with the following bitmask (*the first flag indicates whether the field describes a directory or a file*):

d	r	w	x	r	-	x	r	-	-
	READ	WRITE	EXEC	READ		EXEC	READ		
FILETYPE DIRECTORY	OWNER			GROUP			USER		
	4	2	1	4	0	1	4	0	0
	7			5			4		

In the above example, the file owner has full rights over the file, the members of the file's group have read and execute rights, and all other users have only read rights.

In the console mode of Linux, we can view the rights of the elements in the current directory using the command:

```
ls -la
```

```
sysadmin@ubuntu:~$ ls /usr/bin -al
total 103716
drwxr-xr-x  2 root  root    32768 Jun  1 15:29 .
drwxr-xr-x 10 root  root    4096 May  8 15:48 ..
-rwxr-xr-x  1 root  root   54772 Oct  4 2017 [
-rwxr-xr-x  1 root  root    96 Oct  4 2017 2to3-3.6
-rwxr-xr-x  1 root  root   22008 Sep 15 2017 aa-enabled
-rwxr-xr-x  1 root  root   22104 Sep 15 2017 aa-exec
-rwxr-xr-x  1 root  root   9984 Apr 28 2017 acpi_listen
-rwxr-xr-x  1 root  root   3452 Jul 20 2017 activate-global-python-argcomp
lete3
-rwxr-xr-x  1 root  root    6473 Oct 17 2017 add-apt-repository
-rwxr-xr-x  1 root  root   21916 Aug 14 2017 addpart
lrwxrwxrwx  1 root  root    24 Oct  1 2017 addr2line -> i686-linux-gnu-ad
dr2line
-rwxr-xr-x  1 root  root    2555 Oct 24 2017 apport-bug
-rwxr-xr-x  1 root  root   13348 May 11 01:27 apport-cli
lrwxrwxrwx  1 root  root    10 May 11 01:27 apport-collect -> apport-bug
-rwxr-xr-x  1 root  root   1849 May 11 01:27 apport-unpack
lrwxrwxrwx  1 root  root    6 Dec 13 2016 apropos -> whatis
-rwxr-xr-x  1 root  root   13732 Oct 26 2017 apt
```

In Linux, there are defined additional special modes for files and directories. One of these modes is **SUID** (short for: Set-User IDentification), which is identified in the permissions mask with **(s)** instead of **(x)** at the owner level. When a file

with executable code is in this mode and is executed, the created processes and resources will belong to the owner of the file, not to the user who started the application.

Another special mode is **SGID** (short for: Set-Group IDentification), identified with `(s)` instead of `(x)` at the group level. When a file with executable code is in this mode and is executed, the created processes and resources will belong to the group of the file owner, not to the user who initiated the application. When a directory is in **SGID** mode, files created within it will default to belonging to the group of the parent directory. The **Sticky bit** mode, identified with `(s)` instead of `(x)` at the general level, is often used for shared directories.

The **Sticky bit** mode, which is identified with `(s)` instead of `(x)` at the general level, is often used for shared directories. When a directory is in this mode, users have the right to read and execute files of other users, but they cannot delete or rename them.

Manage Users and Groups

The table below shows commands from the operating system used for managing **users** and **groups**:

Command	Explanation
id	Prints real and effective user and group IDs
chmod	Changes the file mode bit
umask	Sets the mask for creating file mode
chown	Changes the owner of the file and the group
chgrp	Changes the group ownership
passwd	Changes the user password

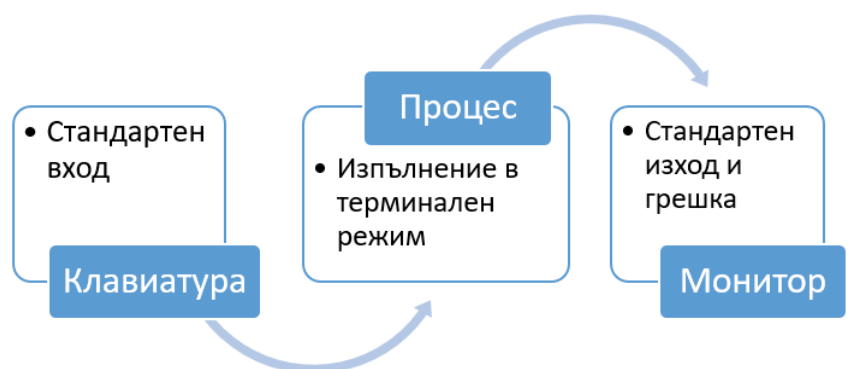
Standart Streams

In general, computer programs receive input data (*keyboard, file, etc.*), process it, and produce output results in the form of output data (*files, text, image, etc.*). For standard console applications, the input data is entered via the computer's keyboard, and the output data is displayed on a text screen.

In this model of input and output of data, it is not known in advance how much and what type of data will be received from users and correspondingly output as a result. This type of input and output is called text **streams**.

Every program that we run in the command line in Linux is automatically connected to three streams of data:

stream	information
STDIN (0)	Standard input (data provided to the program, by default from the keyboard)
STDOUT (1)	Standard output (data printed by the program, by default to the text terminal)
STDERR (2)	Standard error (for error messages, by default to the text terminal)



Redirecting and Pipelines

Linux supports various tools for working with input-output streams, for example:

- The operator **greater than** `>` redirects the output of programs to a file instead of printing it on the screen. If the file already exists, its content will be deleted and a version with the new content will be saved.
- The operator **double greater than** `>>` redirects the output and appends it to a file if it already exists.
- The operator **pipe** `|` redirects the output of the program on the left as input to the program on the right.

```
sysadmin@ubuntu:~$ ls > file1.txt
sysadmin@ubuntu:~$ cat file1.txt
course
file1.txt
sysadmin@ubuntu:~$ ls >> file1.txt
sysadmin@ubuntu:~$ cat file1.txt
course
file1.txt
course
file1.txt
sysadmin@ubuntu:~$ ls | head -3 | tail -1
course
```

In the above example, we create a text file `file1.txt`, which contains a list of the files in the current directory. Note that the file we created is also in the list. First, the file is created, then the command is executed and the output is saved in it.

Next, we add the listing of the current directory again to the same file.

In the third example, we output the list of files in the current directory, redirect it to the command `head`, which takes the first three lines, which are then redirected to the command `tail`, which keeps only the last line.

Nano Text Editor

Text files are a commonly used means for storing and exchanging information in Linux. Because of this, almost every distribution offers tools to work with them. In our working environment, we will use the text editor **nano**:

```
GNU nano 2.8.6      File: readme.txt      Modified
Nano is another editor, an enhanced free Pico clone.

Fast operations:
---
select = drag mouse
paste  = right mouse button

^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File ^\ Replace  ^U Uncut Text ^T To Spell ^_ Go To Line
```

Some of the main features of **nano** are:

File Control

Key	Explanation
nano readme.txt	Opens or creates a file readme.txt
Ctrl-o Y Enter	Saves the changes
Ctrl-r Alt-f	Opens a new file
Alt->	Switches to the next open file
Alt-<	Switches to the previous open file
Ctrl-x	Exits the editor

Navigation in the Content

Key	Explanation
Ctrl-a	Move to the beginning of the current line
Ctrl-e	Move to the end of the current line
Ctrl-v	Scroll down a page
Ctrl-y	Scroll up a page
Alt-	Move to the beginning of the file
Alt-/	Move to the end of the file
Alt-g	Position to the desired line in the file

Copying and Pasting

Key	Explanation
Alt-a	Select a block for copying or pasting
Alt-a Alt-^	Copy (Copy) the selected block to the clipboard
Alt-a Ctrl-k	Cut (Cut) the selected block to the clipboard
Ctrl-k	Cut (Cut) from the cursor position to the end of the line
Ctrl-u	Paste (Paste) the content from the clipboard at the current position

Search and Replace

Key	Description
Ctrl-w	Search for text
Alt-w	Repeat last search
Alt-r	Search and replace

References:

- [The Beginner's Guide to Nano, the Linux Command-Line Text Editor](#)
- [Nano text editor command cheatsheet](#)

Linux Basics Exercise

Practice the materials from the topic by completing the tasks presented below.

1. FTP

Using the program **FileZilla**, establish a remote connection to the working server at address **46.10.253.12**. Upload the file **LSPp1.pdf** to the user's home folder, which you can download from the Internet at the address https://github.com/dimitarminchev/LSP/presentations/01_Linux_Basics.pdf.

For the username and password, use those provided by the instructor. Alternatively, you can log in with the username **students** and password **password123**.

2. SSH

Using the program **putty**, establish a remote connection to the working server at address **46.10.253.12**.

For the username and password, use those provided by the instructor. Alternatively, you can log in with the username **students** and password **password123**.

3. Terminal

1. Check if the file **LSPp1.pdf** is present in the user's home folder.
2. Change your password.
3. In your home folder **~**, create a new folder **exercises** for the exercises and a subdirectory **day01** in it for this lecture.
4. Go to the last folder **day01** and create a file **listing.txt** there, which contains a list of files from the folder **/sbin**.
5. Open the last file with the text editor **nano** and add your names at the beginning.
6. In your home directory, create a link to the directory **/home/students/**.

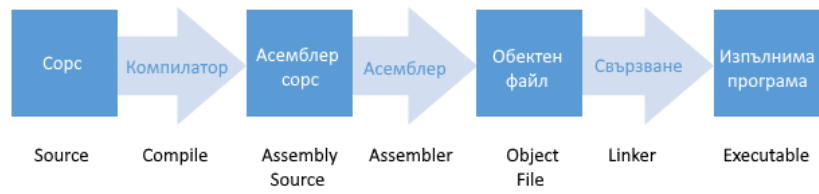
Example:

```
ls /home/students/LSPp1.pdf
passwd
cd ~
mkdir exercises
mkdir exercises/day01
cd exercises/day01/
ls -l /sbin/ > listing.txt
nano listing.txt
cd ~
ln -s /home/students/ shared
```


Chapter 2 – Programming

- [Compiling](#)
- [Compiler](#)
- [Source Code](#)
- [Assembly](#)
- [Object](#)
- [Binary](#)
- [Execute](#)
- [Libraries](#)
- [Archiver](#)
- [Object Files](#)
- [Static and Dynamic Library](#)
- [Library Example](#)
- [Debugging](#)
- [Programmng Exercise](#)

Compiling



Reference: [Compiling, assembling, and linking](#)

Compiler

- The GCC compiler was developed under the GNU project and supports various programming languages.
- It was originally called GNU C Compiler when it only worked with the C programming language.
- We will use it to make our programs executable.

	
Developer(s)	GNU Project
Initial release	May 23, 1987; 30 years ago ^[1]
Stable release	8.1 ^[2] (6.x also supported) / May 2, 2018; 15 days ago
Repository	https://gcc.gnu.org/viewcvs/gcc/ 
Written in	C; and C++ since June 1, 2010; 7 years ago ^[3]
Operating system	Cross-platform
Platform	GNU
Type	Compiler
License	GNU GPL 3+ with GCC Runtime Library Exception ^[4]
Website	gcc.gnu.org 

Reference: https://en.wikipedia.org/wiki/GNU_Compiler_Collection

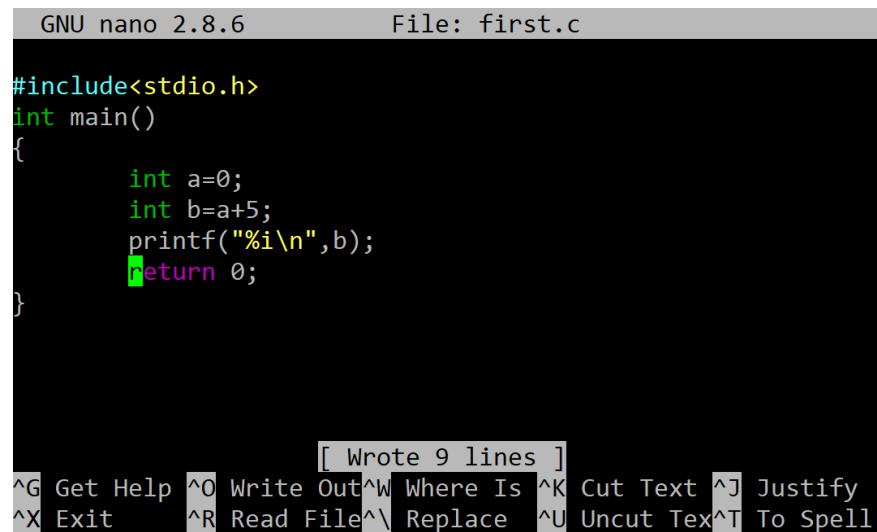
Source Code

Using the text editor **nano**, write the source program code in the file **first.c**.

```
#include<stdio.h>

int main()
{
    int a=0;
    int b=a+5;
    printf("%i\n",b);
    return 0;
}
```

View of the source code in the text editor **nano**:



```
GNU nano 2.8.6      File: first.c

#include<stdio.h>
int main()
{
    int a=0;
    int b=a+5;
    printf("%i\n",b);
    return 0;
}

[ Wrote 9 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify
^X Exit      ^R Read File ^\\ Replace   ^U Uncut Tex ^T To Spell
```

Assembly

Using the **gcc** compiler, compile to **assembly** by executing the following command:

```
gcc -S first.c -o first.s
```

The result is saved in a file **first.s**, which should look like this:

```
GNU nano 2.8.6      File: first.s
.file "first.c"
.section .rodata
.LC0:
.string "%i\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
leal 4(%esp), %ecx
.cfi_def_cfa 1, 0
andl $-16, %esp
pushl -4(%ecx)
pushl %ebp
.cfi_escape 0x10,0x5,0x2,0x75,0
movl %esp, %ebp
pushl %ebx
pushl %ecx
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify
^X Exit ^R Read File ^\ Replace ^U Uncut Text ^T To Spell
```

Object

Using the **gcc** compiler, compile to an **object** file by executing the following command:

```
gcc -c first.c -o first.o
```

The result is saved in a file **first.o**, which should look like the following:

[illegible]

Binary

Using the compiler **gcc**, compile to the executable program **binary** by executing the following command:

```
gcc first.c -o first
```

The result is recorded in a file **first**, which should look as follows:

```
GNU nano 2.8.6                               File: first
^?ELF^A^A^A@^@^@^@^@^@^@^@^@C^@^C^@A^@^@^@^@C^@^@4^@^@^@^@
^@^@B^@N^@^@^@^@^@^@^@^@H^@^@^@^@^@^@H^@^@^@^@_ ^@^@
^@^@HPQ^@P^@] ^@^@^@^@' ^@^@^@^@U^@S^@W^@^@^@^@O^@ ^@^@^@^@D^@^@^@^@
^@^@A^@[ ^C;8^@^@^@F^@^@^@^@T^@^@^@^@x^@^@^@Y^@^@^@^@k^@
^@^@^@^@E^@^@^@Y^@^@^@^@^@^@^@^@^@^@D^@^@^@Z^@^@^@^@^@^@^@
^@^@^@^@^@^@K^@^@^@P^@^@^@U^@^@^@^@^@^@^@^@C^@^@^@^@_ ^@^@
^@^@^@^@^@| ^C^@^@^@^@^@^@C^@^@K^@^@^@^@^@^@C^@^@^@^@^@^@C^@
^@^@^@^@^@^@C^@^@^@^@^@^@C^@^@N^@^@^@^@^@^@E^@^@^@^@^@^@C^@
[ Read 8 lines (Converted from Mac format) ]
^G Get Help   ^O Write Out^W Where Is  ^K Cut Text  ^J Justify
^X Exit       ^R Read File^\ Replace    ^U Uncut Tex^T To Spell
```

Execute

To start the executable program, use the following command line:

```
./first
```

The result of the execution should look like this:

```
sysadmin@ubuntu:~/tmp$ ./first  
5  
sysadmin@ubuntu:~/tmp$
```


Libraries

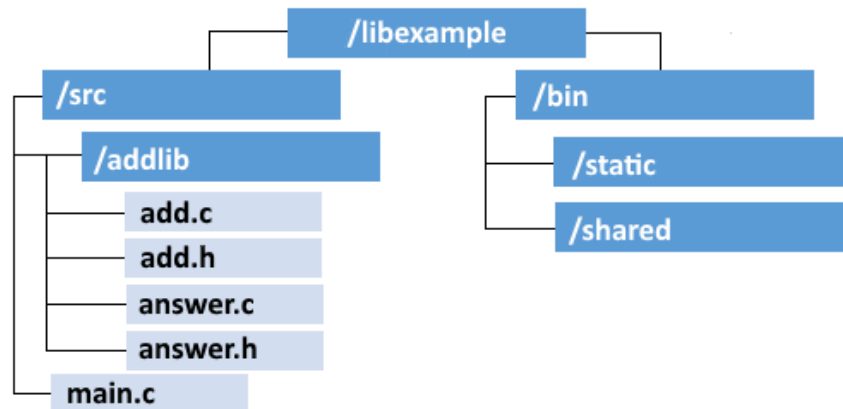
Libraries represent a collection of compiled objects in a single file.

Their advantages are: **reusability of components** (*using one shared library takes up less disk space*), **version management** (old and new versions coexist simultaneously on one Linux system), **component specialization** (*developers can focus their core competency in one library*).

The types of libraries are: **static** (*object code in a linked library, which becomes part of the application*) and **dynamic** (*shared objects, dynamically linked at runtime*).

Structure of a library

The figure below shows a sample structure of a library:



Directory Structure

Use the following commands to create the directory structure:

```
mkdir libexample
mkdir libexample/src
mkdir libexample/src/addlib
mkdir libexample/bin
mkdir libexample/bin/static
mkdir libexample/bin/shared
```

Reference: [Creating a shared and static library with the gnu compiler gcc](#)

Library files

In the folder `libexample/src/addlib/`, create a file `add.c` with the following content:

```
#include <stdio.h>

int gSummand;

void setSummand(int summand) {
    gSummand = summand;
}

int add(int summand) {
    return gSummand + summand;
}

void __attribute__((constructor)) initLibrary(void) {
    printf("Library is initialized\n");
    gSummand = 0;
}

void __attribute__((destructor)) cleanUpLibrary(void) {
    printf("Library is exited\n");
}
```

In the folder `libexample/src/addlib/`, create a file `add.h` with the following content:

```
void setSummand(int summand);
int add(int summand);
```

In the folder `libexample/src/addlib/`, create a file `answer.c` with the following content:

```
#include "add.h"

int answer() {
    setSummand(20);
    return add(22); // 42 = 20 + 22
}
```

In the folder `libexample/src/addlib/`, create a file `answer.h` with the following content:

```
int answer();
```

In the folder `libexample/src/`, create a file `main.c` with the following content:

```
#include <stdio.h>
#include "addlib/add.h"
#include "addlib/answer.h"
int main(int argc, char* argv[])
{
    setSummand(5);
    printf("5 + 7 = %d\n", add(7));
    printf("And the answer is: %d\n", answer());
    return 0;
}
```

Archiver

The archiver allows: creating, modifying, and extracting archives. With its help, you can create libraries. For example, the following command creates a library:

```
ar rcs <library-name>.a <module-1>.o <module-2>.o ...
```

To list the files from the library, use the following command:

```
ar -t <library-name>.a
```

Object Files

To create the object files (directory libexample) use the following command:

```
gcc -c src/main.c -o bin/main.o
```

To create the object files intended for a static library, use:

```
gcc -c src/addlib/add.c -o bin/static/add.o  
gcc -c src/addlib/answer.c -o bin/static/answer.o
```

To create the object files intended for a shared library, use:

```
gcc -c -fPIC src/addlib/add.c -o bin/shared/add.o  
gcc -c -fPIC src/addlib/answer.c -o bin/shared/answer.o
```

Static and Dynamic Library

Combine the files of the objects into a single static library/archive using the command:

```
ar rcs bin/static/libadd.a bin/static/add.o bin/static/answer.o
```

Link the static main.o with the library using the command:

```
gcc bin/main.o -Lbin/static -ladd -o bin/statically-linked
```

For a shared library in GCC ending with .so instead of .a, use the command:

```
gcc -shared bin/shared/add.o bin/shared/answer.o -o bin/shared/libadd.so
```

Connect dynamically to the shared library using the command:

```
gcc bin/main.o -Lbin/shared -ladd -o bin/use-shared-library
```

Move the shared library to the default location:

```
sudo mv bin/shared/libadd.so /usr/lib
sudo chmod u=rwx,go=rx /usr/lib/libadd.so
```

Use the shared library through **LD_LIBRARY_PATH**:

```
LD_LIBRARY_PATH=$(pwd)/bin/shared bin/use-shared-library
```

Library Example

In the `libexample/` folder, create a file `test.c` with the following content:

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

int main()
{
    // Load the dynamic library:
    void* lib_add = dlopen("bin/shared/libadd.so", RTLD.LAZY | RTLD.GLOBAL);

    // Declare the function pointers:
    void (*fptr_setSummand)(int);
    int (*fptr_add)(int);

    // Get the pointers to the functions within the library:
    fptr_setSummand = dlsym(lib_add, "setSummand");
    fptr_add = dlsym(lib_add, "add");

    // Call the function via the function pointer:
    fptr_setSummand(42);
    int result = fptr_add(7);
    printf("Result: %d\n", result);

    return 0;
}
```

Compile and run the test program using the commands:

```
gcc test.c -ldl -o test
./test
```

As a result, if executed successfully, you should see the following three lines:

```
Library is initialized
Result: 49
Library is exited
```

Debugging

Before we get acquainted with the debugging possibilities, let's write a short program to find the sum of numbers: **suma.c**

Summing Numbers

Initialize a variable **sum=0**. Print the value of **argc**. Print the values in **argv**. The numbers counter **num_count** is the second value in **argv**. Read a number from the keyboard **num_count** times and add it to the sum. Print the value of the obtained sum.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    int numbers_count = 0, sum = 0;
    char temp_str[50];
    printf("The value of argc: %d\n", argc);
    printf("The value(s) in argv:\n");
    for(int i = 0; i < argc; i++) {
        printf("> argv[%d]=%s\n", i, argv[i]);
    }
    numbers_count = atoi(argv[1]);
    printf("Enter %d numbers:\n", numbers_count);
    for(int i = 0; i < numbers_count; i++) {
        scanf("%s", temp_str);
        sum += atoi(temp_str);
    }
    printf("Total sum is %d\n", sum);
}
```

```
$ ./suma 4
The value of argc: 2
The value(s) in argv:
> argv[0]=./suma
> argv[1]=4
Enter 4 numbers:
1
2
3
4
Total sum is 10
```

Debugging

To compile the source code of the program with debugging enabled:


```
gcc suma.c -o suma
```

To start the debugger:

```
gdb suma
```

To set a breakpoint where the program will stop execution:

```
b [function name, line number]
```

To start the program:

```
r [command line arguments]
```

The table provides the shortcut key combinations useful when working with the debugger:

Key	Information
h	Help
n	Step forward one block of code
s	Step forward one line of code
p [variable]	Prints the value of the variable variable
info locals	Prints the values of all local variables
bt	Shows the sequence of functions called up to this execution point
q	Exit

Reference: [Introduction to GDB](#) и [Harvard University CS50](#)

Programmng Exercise

Practice the materials from the topic by implementing the projects presented below.

Multi Add

Write a program **multi-add.c** that takes one or more numbers as input arguments, then calculates the total sum and prints the answer to standard output. Compile the program and run it. Compile. Try debugging using a debugger.

Reverse Test

Write a program **reverse-test.c** that reads a string from the keyboard and prints it in reverse. To reverse the input string, you need to use the **reverse** library, which is provided in the directory **/day02/reverse/**. The function you need to use is:

```
void inplace_reverse(char * str)
```

You have all the resources to compile and link the library statically or you can use the shared library **reverse** from **/usr/lib/libreverse.so**. It is best to try both approaches.

Chapter 3 – File System and Files

- [Files](#)
- [File Systems and Namespaces](#)
- [Working with Files](#)
- [Buffered vs Unbuffered Streams](#)
- [Library for Working with Files](#)
- [Opening and Closing a File](#)
- [Reading File Contents](#)
- [Reading and Printing a File](#)
- [Reading and Writing to a File](#)
- [Writing Sentences to a File](#)
- [Seeking in Files and Sparse Files](#)
- [Create a File with a Hole](#)
- [File Locking](#)
- [Lock and Write There](#)
- [File Handling Exercise](#)

Files

One of the basic principles in Unix and Linux states: **Everything is a file!**. Data files, directories, information about hardware devices in the system, information about processes, or important system configuration parameters are all represented as files.

The following types of files exist:

- **inode** - a data structure that describes a file system object.
- Files are always opened from user space with a name.
- The pair **filename** and **inode** is called a link.
- **Regular files** = bytes of data.
- **Directories** = mapping between file names and inodes (links).
- **Hard links** = multiple links connect different names to the same inode.
- **Symbolic links** = like regular files that contain the full path of the linked files.
- **Special files** = block device files, character device files, named pipes, Unix domain sockets.

File Systems and Namespaces

Linux provides a global and unified namespace of files and directories with root /. A file system is a collection of files and directories in a formal and valid hierarchy. File systems can be added (mounted) separately and removed (unmounted) from the global namespace of files and directories. Some directories are special, for example **/dev** and **/proc**.

```
sysadmin@ubuntu:/dev$ ls
autofs          kmsg           sg0            tty30          tty61          ttyS5
block           kvm            sg1            tty31          tty62          ttyS6
bsg             lightningvm    shm            tty32          tty63          ttyS7
btrfs-control   log            snapshot       tty33          tty7           ttyS8
bus             loop0          snd            tty34          tty8           ttyS9
cdrom           loop1          sr0            tty35          tty9           ubuntu-vg
cdrw            loop2          stderr         tty36          ttyprintk      uhid
char            loop3          stdin          tty37          ttyS0          uinput
console         loop4          stdout         tty38          ttyS1          urandom
core            loop5          tty            tty39          ttyS10         userio
cpu             loop6          tty0           tty4           ttyS11         vcs
cpu_dma_latency loop7          tty1           tty40          ttyS12         vcs1
cuse            loop-control   tty10          tty41          ttyS13         vcs2
disk            mapper         tty11          tty42          ttyS14         vcs3
dm-0            mcelog         tty12          tty43          ttyS15         vcs4
```

```
sysadmin@ubuntu:/proc$ ls
1      18      25210  42    59     766    diskstats  net
10     18956   25211  43    6       774    dma        pagetypeinfo
1044   19      25212  431   60      78     driver     partitions
1047   2       25298  432   61      783    execdomains sched_debug
11     20     25299  437   62     7922    fb         schedstat
12     20693  25327  439   63      8      filesystems scsi
124    21      255    44    64      80     fs         self
125    21753  256    440   65     800     interrupts slabinfo
128    21816  26     441   66      81     iomem      softirqs
12841  21934  27     442   67     815     ioports    stat
129    22     27640  445   68      82     ipmi       swaps
13     23259  28     447   69     822     irq        sys
130    24     29998  448   7       855     kallsyms   sysrq-trigger
131    240    30     449   70     882     kcore      sysvipc
```

The directory structure in Linux is defined by a standard that can be found at <http://www.pathname.com/fhs/>.

Working with Files in C

Before a file can be read or written, it must be opened. The kernel maintains a list of open files called the file table. This table is indexed by non-negative integers known as file descriptors (often abbreviated as fds). Each entry in the list contains information about an open file, including a pointer to an in-memory copy of the file's backing inode and associated metadata, such as file position and access modes. Both user space and kernel space use file descriptors as unique "cookies" during operation. Opening a file returns a file descriptor, and subsequent operations (read, write, etc.) accept the file descriptor as a primary argument.

Basic methods for working with files are presented in the table below:

Method	Description
<code>fopen()</code>	Opens a file and returns a file descriptor
<code>fread()</code>	Reads data from a file
<code>fwrite()</code>	Writes data to a file
<code>fflush()</code>	Flushes the stream
<code>close()</code>	Closes the file descriptor
<code>fseek()</code>	Sets the file descriptor position to a given value
<code>fcntl()</code>	Manipulates the file descriptor (e.g., for locking)
<code>errno</code>	Number of the last error

Buffered vs Unbuffered Streams

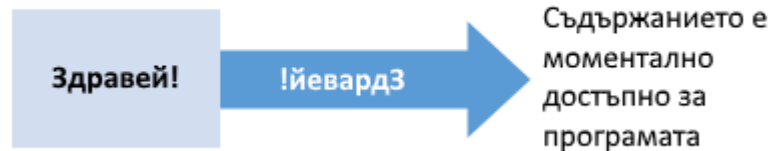
A stream represents the transmission of data from one side to another. A file represents a place for storing data on disk. A file uses streams to store and load data. A buffer is used temporarily to store data from the stream.

Characters written to or read from an **unbuffered stream** are transmitted individually to or from the file as soon as possible. Characters written to or read from a **fully buffered stream** are transmitted to or from the file in blocks of arbitrary size. Characters written to a **line buffered stream** are transmitted to the file in blocks when a new line appears.

Low-Level File Operations - Unbuffered Stream

The methods `open()`, `read()`, `write()`, `lseek()` are part of the `unistd.h` library. They work with the file descriptor (`INT`) and treat the input/output stream as binary data.

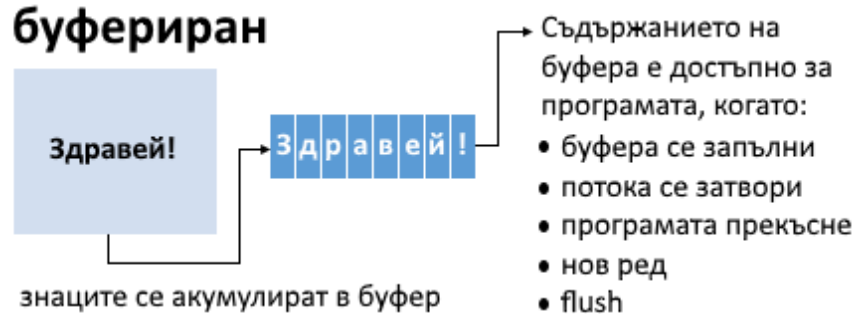
небуфериран



Low-Level File Operations - Buffered Stream

The methods `fopen()`, `fread()`, `fwrite()`, `fseek()` are part of the `stdio.h` library. They use the `FILE` object type and treat the input/output stream as text. Read/write operations on the accumulated buffer can be forced using the `fflush()` method.

буфериран



Library for Working with Files

The **unistd.h** library contains methods for working with files:

```
#include<unistd.h>

int open(const char *name, int flags);

int open(const char* name, int flags, mode_t mode);

int close();
```

The flag parameter (**flag**) is a bitmask defining the following:

flag	Information
O_APPEND	The file will be opened in append mode.
O_ASYNC	SIGIO is generated when it can be read or written.
O_CREAT	If the file does not exist, it creates it.
O_DIRECT	Opens the file for direct input/output.
O_DIRECTORY	If the name is not a directory, it gives an error
O_EXCL	If O_CREAT and the file exists, it gives an error.
O_LARGEFILE	Opens a file larger than 2 GB
O_NOCTTY	This flag is not used
O_NOFOLLOW	If name is a symbolic link, it gives an error
O_NONBLOCK	If possible, opens the file in non-blocking mode.
O_SYNC	The file will be opened for synchronous Input/Output
O_TRUNC	If the file exists, it is truncated to zero length

In case the **O_CREAT** flag is set, the mode is a bitmask as follows:

S_IRWXU	S_IXUSR	S_IWGRP	S_IROTH
S_IRUSR	S_IRWXG	S_IXGRP	S_IWOTH
S_IWUSR	S_IRGRP	S_IRWXO	S_IXOTH

Opening and Closing a File

- Initialize the variables
- Open the file for read-only and get a file descriptor
- If the file descriptor is -1, print an error message
- Otherwise, print the value of the file descriptor
- Close the file
- If closing fails, output an error message

openclose.c

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char * argv[])
{
    int fd;
    fd = open ("/proc/self/environ", O_RDONLY);
    if (fd == -1) {
        printf("ERROR opening 'environ'!\n");
    } else {
        printf("File Descriptor of 'environ' = %d\n", fd);
    }

    // write/read commands to be added here

    if (close (fd) == -1)
        printf("ERROR closing the file!\n");
}
```

Reading File Contents

The **unistd.h** library contains methods for working with files:

```
#include<unistd.h>

ssize_t read(int fd, void *buf, size_t len);
```

- Each call reads **len** number of bytes into **buf** from the current position of the file specified in the file descriptor **fd**.
- On success, it returns the number of bytes written to **buf**.
- On error, it returns -1, and error information is written to **errno**.

Actually, the read method can lead to many possibilities:

- Returns a value equal to **len** and all read bytes are stored in **buf**. The results are as expected.
- Returns a value less than len but greater than zero. The read bytes are stored in **buf**. This can happen because a signal interrupted the read in the middle, an error occurred in the middle of reading, more than zero but less than the value of len bytes are available, or EOF is reached before len bytes are read. Reissuing the read (with correspondingly updated **buf** and **len** values) will read the remaining bytes into the rest of the buffer or indicate the cause of the problem.
- Returns a value of 0. This indicates **EOF**. There is nothing to read.
- The method call blocks because no data is available. This will not happen in non-blocking mode.
- Returns a value of -1 and the error is set to **EINTR**. This indicates that a signal was received before any bytes were read.
- Returns a value of -1 and the error is set to **EAGAIN**. This indicates that the read would block because no data is currently available. This only happens in non-blocking mode.
- Returns a value of -1 and the error is set to a value other than **EINTR** or **EAGAIN**. This indicates a more serious error.

Reading and Printing a File

Initialize the variables. Open the file **readfile.c**, print an error if there's a problem. While reading from the file returns a length different from 0. If the length is -1 and the error is **EINTR**, try to read again. If the length is -1 and the error is not **EINTR**, output an error and stop. Otherwise, print the buffer. Close the file, print an error if there's a problem.

readfile.c

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#define BUF_SIZE 1000000
int main(int argc, char *argv[])
{
    int fd = open("./readfile.c", O_RDONLY);
    if (fd == -1)
    {
        printf("ERROR opening the file!\n");
        return -1;
    }
    ssize_t len;
    char buf[BUF_SIZE];
    while ((len = read(fd, buf, BUF_SIZE - 1)) != 0)
    {
        if (len == -1)
        {
            if (errno == EINTR) continue;
            printf("ERROR reading the file!\n");
            break;
        }
        printf("%s", buf);
    }
    if (close(fd) == -1)
    {
        printf("ERROR closing the file!\n");
        return -2;
    }
    return 0;
}
```

Reading and Writing to a File

```
#include<unistd.h>

ssize_t write(int fd, const void *buf, size_t count);
```

The **unistd.h** library contains methods for working with files:

- Writes **count** number of bytes, starting from **buf** to the current position in the file specified by the file descriptor **fd**.
- On success, it returns the number of written bytes and updates the current position in the file.
- On error, it returns -1, and the information is written to **errno**.

Note:

When a call to `write( )` returns, the kernel has copied the data from the supplied buffer into a kernel buffer, but there is no guarantee that the data has been written out to its intended destination. Indeed, `write` calls return much too fast for that to be the case. The disparity in performance between processors and hard disks would make such behavior painfully obvious.

Instead, when a user-space application issues a `write( )` system call, the Linux kernel performs a few checks, and then simply copies the data into a buffer. Later, in the background, the kernel gathers up all of the "dirty" buffers, sorts them optimally, and writes them out to disk (a process known as `writeback`). This allows `write` calls to occur lightning fast, returning almost immediately. It also allows the kernel to defer writes to more idle periods, and batch many writes together.

The delayed writes do not change POSIX semantics. For example, if a read is issued for a piece of just-written data that lives in a buffer and is not yet on disk, the request will be satisfied from the buffer, and not cause a read from the "stale" data on disk. This behavior actually improves performance, as the read is satisfied from an in-memory cache without having to go to disk. The read and write requests interleave as intended, and the results are as expected—that is, if the system does not crash before the data makes it to disk! Even though an application may believe that a write has occurred successfully, in this event, the data will never make it to disk.

Writing Sentences to a File

Create an empty file **sentences.txt**, in case of failure print an error and exit. Do the following 100 times: call **getSentence()** to get new text in the buffer and its length, write the buffer to the file and record the number of bytes. If the number of written bytes is -1, print an error and exit. Close the file, output an error in case of failure.

writefile.c

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/time.h>
#include <string.h>
#define BUF_SIZE 1000000
long getMicrotime(){
    struct timeval currentTime;
    gettimeofday(&currentTime, NULL);
    return currentTime.tv_sec * (int)1e6 + currentTime.tv_usec;
}
int getSentence(char *buf) {
    sprintf(buf, "%ld> This is a new sentence.\n", getMicrotime());
    return strlen(buf);
}
int main(int argc, char *argv[])
{
    int fd;
    fd = open("./sentences.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        printf("ERROR opening the file!\n");
        return -1;
    }
    char buf[BUF_SIZE];
    for (int i=0; i < 100; i++)
    {
        int text_len = getSentence(buf);
        ssize_t nr = write(fd, buf, text_len);
        if (nr == -1)
        {
            printf("ERROR writing to the file!\n");
            break;
        }
    }
    if (close(fd) == -1)
    {
        printf("ERROR closing the file!\n");
        return -2;
    }
    printf("SUCCESS!\n");
    return 0;
}
```

Seeking in Files and Sparse Files

```
#include<stdio.h>

FILE *fopen(const char *pathname, const char *mode);

size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);

int fseek(FILE *stream, long offset, int whence);

int fclose(FILE *stream);
```

- The behavior of **fseek()** depends on the argument: **SEEK_CUR**, **SEEK_END**, or **SEEK_SET**.
- The method returns the new file position on success.
- On error, it returns -1, and error information is written to **errno**.
- Seeking past the end of a file and then writing to it causes **holes** filled with zeros.
- Files with holes are called **sparse files**.
- Holes do not occupy physical disk space.
- **du** = estimate of the disk space used by the file.

Argument	Description
SEEK_CUR	Takes the value of the current file position (fd) plus the value POS , which can be negative, zero, or positive. POS of zero returns the value of the current file position.
SEEK_END	The value of the current file position (fd) becomes equal to the file length plus the value POS , which can be negative, zero, or positive. POS of zero sets the offset to the end of the file.
SEEK_SET	The value of the current file position (fd) becomes equal to the value POS . POS of zero sets the offset to the beginning of the file.

More information: [Sparse Files](#)

Create a File with a Hole

- Initialize the variables
- Create a file using the first argument passed to the program as its name
- If it doesn't open, output an error and exit
- Write the file name to the file
- Skip 16777216 bytes forward in the file
- Write the file name to the file again
- Close the file

makesparse.c

```
#include <stdio.h>
#include <string.h>

#define BIG_SIZE 0x1000000

int main(int argc, char * argv[])
{
    FILE * f;
    f = fopen(argv[1], "w");
    if (f == NULL)
    {
        printf("ERROR creating file: %s", argv[1]);
        return 1;
    }
    fwrite(argv[1], 1, strlen(argv[1]), f);
    fseek(f, BIG_SIZE, SEEK_CUR);
    fwrite(argv[1], 1, strlen(argv[1]), f);
    fclose(f);
}
```


File Locking

Use the **fcntl()** method, which provides a pointer to the **flock** structure. This call manipulates the file descriptor **fd**, depending on the command **cmd**:

- To lock a block of a file, use **F_SETLK**.
- If the block is already locked, use **F_GETLK** to get information about the locking process.
- To unlock a block of a file, use **F_SETLK**, but set **flock.l_type = F_UNLCK**.

```
#include<stdio.h>

FILE *fopen(const char *pathname, const char *mode);

size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);

int fseek(FILE *stream, long offset, int whence);

int fclose(FILE * stream);
```

The flock structure

```
struct flock {

    ...

    short l_type;    // F_RDLCK, F_WRLCK, F_UNLCK
    short l_whence;  // SEEK_SET, SEEK_CUR, SEEK_END

    off_t l_start;   // Starting offset for lock
    off_t l_len;     // Number of bytes to lock

    pid_t l_pid;     // PID of process blocking our lock
    ...
}
```

Lock and Write There

Open for writing or create `./testlocks.txt`. While locking 64 bytes from **offset** is unsuccessful: get and print information about the locking process and move **offset** by another 64 bytes. Move **offset** bytes from the beginning of the file and write information about the current process. Wait for the **Enter** key to be pressed. Unlock the 64 locked bytes, output an error on failure. Close the file.

lockfile.c

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
int main(int argc, char * argv[])
{
    char str[64];
    memset(str, 32, 64);
    struct flock fi;
    sprintf(str, "Stored by process %i", getpid());
    int fd = open("testlocks.txt", O_RDWR|O_CREAT);
    fi.l_type = F_WRLCK;
    fi.l_whence = SEEK_SET;
    fi.l_start = 0;
    fi.l_len = 64;
    int off = 0;
    while (fcntl(fd, F_SETLK, &fi) == -1)
    {
        fcntl(fd, F_GETLK, &fi);
        printf("bytes %i - %i blocked by process %i\n", off, off+64, fi.l_pid);
        off += 64;
        fi.l_start = off;
    }
    lseek(fd, off, SEEK_SET);
    write(fd, str, strlen(str));
    getchar();
    fi.l_type = F_UNLCK;
    if (fcntl(fd, F_SETLK, &fi) == -1) printf("ERROR while blocking!\n");
    close(fd);
}
```

File Handling Exercise

Practice the materials from this topic by implementing the projects presented below.

File Manipulator

Write a program **file-manipulator.c** that requires 3 arguments: **words_count**, **min_length**, **max_length**. The program should generate a file called **file-manipulator.log** that contains **words_count** number of words (*separated by spaces*). To generate the words, create a function **random_word_generator** (*the function will not generate real words, but rather random sequences of characters*):

```
char* random_word_generator(int min_length, int max_length);
```

Then the program should print the generated file to the screen.

After the program creates the file **file-manipulator.log** and before writing to it, it should lock the first 100 bytes of the file. When the program prints the generated words, it should wait for the **Enter** key to be pressed, then unlock the file.

Accordingly, if the file is already locked by another process, the program should notify about this and wait for the **Enter** key to be pressed before trying again (*to lock, generate, and write*).

Compile and run several times for testing.

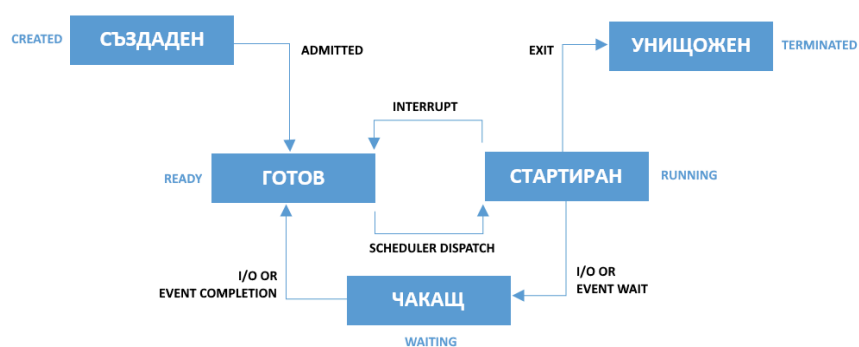
Chapter 4 – Processes

- [Processes](#)
- [Process Management](#)
- [Methods for Working with Processes](#)
- [Executing a Command](#)
- [Display File Source Code](#)
- [Starting a Child Process](#)
- [Print Process IDs](#)
- [Waiting for and Terminating a Child Process](#)
- [Execute and Print Status](#)
- [Zombies and Simple Signal Handling](#)
- [Zombie Game](#)
- [Process Exercise](#)

Processes

- Processes are object code in execution: active, alive, running programs.
- Processes consist of data (**data**), resources (**resources**), state (**state**), and a virtualized computer (**virtualized computer**).
- Each process is represented by a unique process identifier (**pid**).
- The process that the kernel starts when there are no other executable processes is the idle process (**idle process**), its identifier is **pid = 0**.
- The process that creates a new process is known as the parent (**parent**), and the new process is known as the child (**child**).
- Each process is owned by a user (**user**) and a group (**group**).

The possible states of a process are shown in the figure below:



Note:

If files are the most fundamental abstraction in a Unix system, processes are the second most fundamental. Processes are object code in execution: active, alive, running programs. But they're more than just object code—processes consist of data, resources, state, and a virtualized computer.

Processes begin life as executable object code, which is machine-runnable code in an executable format that the kernel understands (the format most common in Linux is ELF). The executable format contains metadata, and multiple sections of code and data. Sections are linear chunks of the object code that load into linear chunks of memory. All bytes in a section are treated the same, given the same permissions, and generally used for similar purposes.

The most important and common sections are the text section, the data section, and the bss section. The text section contains executable code and read-only data, such as constant variables, and is typically marked read-only and executable. The data section contains initialized data, such as C variables with defined values, and is typically marked readable and writable. The bss section contains uninitialized global data. Because the C standard dictates default values for C variables that are essentially all zeros, there is no need to store the zeros in the object code on disk. Instead, the object code can simply list the uninitialized variables in the bss section, and the kernel can map the zero page (a page of all zeros) over the section when it is loaded into memory. The bss section was conceived solely as an optimization for this purpose. The name is a historic relic; it stands for block started by symbol, or block storage segment. Other common sections in ELF executables are the absolute section (which contains nonrelocatable symbols) and the undefined section (a catchall).

A process is also associated with various system resources, which are arbitrated and managed by the kernel. Processes typically request and manipulate resources only through system calls.

Resources include timers, pending signals, open files, network connections, hardware, and IPC mechanisms. A process' resources, along with data and statistics related to the process, are stored inside the kernel in the process' process descriptor.

A process is a virtualization abstraction. The Linux kernel, supporting both preemptive multitasking and virtual memory, provides a process both a virtualized processor, and a virtualized view of memory. From the process' perspective, the view of the system is as though it alone were in control. That is, even though a given process may be scheduled alongside many other processes, it runs as though it has sole control of the system. The kernel seamlessly and transparently preempts and reschedules processes, sharing the system's processors among all running processes. Processes never know the difference.

Similarly, each process is afforded a single linear address space, as if it alone were in control of all of the memory in the system. Through virtual memory and paging, the kernel allows many processes to coexist on the system, each operating in a different address space. The kernel manages this virtualization through hardware support provided by modern processors, allowing the operating system to concurrently manage the state of multiple independent processes.

Process Management

The table below shows commands for managing processes in a Linux operating system:

Command	Information
ps	snapshot of current processes.
top	displays Linux processes.
kill	sends a signal to a process.
pgrep, pkill	search or signal processes based on name and other attributes.
killall	kills processes by name.
Ctrl-C	kills the foreground process.
Ctrl-Z	suspends the foreground process.
jobs	shows the status of jobs.
fg	brings the suspended process to the foreground.

More information: <https://www.tecmint.com/linux-process-management/>

Example:

```
$ pgrep -u tecmint top
$ kill 2308
$ pgrep -u tecmint top
$ pgrep -u tecmint glances
$ pkill glances
$ pgrep -u tecmint glances
```


Methods for Working with Processes

The table below shows C methods used in process programming in a Linux operating system:

Command	Information
getpid()	returns the pid identifier of the calling process.
getppid()	returns the pid identifier of the parent process.
execl(), execlp(), execl(), execv(), execvp(), execvpe()	execute a command.
fork()	creates a child process.
exit()	causes normal process termination.
wait(), waitpid(), waitid()	waits for the process to change status.

Executing a Command

```
#include<unistd.h>

int execl (const char *path, const char *arg, ...);
int execlp (const char *file, const char *arg, ...);
int execl_e (const char *path, const char *arg, ..., char *const envp[]);
int execv (const char *path, char *const argv[]);
int execvp (const char *file, char *const argv[]);
int execve (const char *filename, char *const argv[], char *const envp[]);
```

Useful notes:

- **l** and **v** indicate whether arguments are provided via a list or an array (vector).
- **p** means that the user's path is searched for the given file. Commands using the **p** variants can specify only a filename, as long as it is located in the user's path.
- **e** indicates that a new environment is also provided for the new process.

Error Values

On success, the system calls do not return. On failure, the system calls return -1 and set the error to one of the following values:

Стойност	Информация
E2BIG	Общият брой байтове в предоставения аргумент (arg) или средата (envp) е твърде голям.
EACCESS	Процесът няма разрешение за търсене на компонент в пътя; пътя не е обикновен файл; целевият файл не е маркиран като изпълним; или файловата система, на която се намира пътя или файла не е монтиран като изпълним.
EFAULT	Даденият указател не е валиден.
EIO	Възникна грешка вход / изход на ниско ниво (това е лошо).
EISDIR	Крайният компонент в пътя или интерпретаторът е директория.
ELOOP	Системата е срещнала твърде много символични връзки в пътя.
EMFILE	Процесът на извикване достигна своята граница за отворените файлове.
ENFILE	Беше достигнато ограничението за отворените файлове в цялата система.
ENOENT	Целта на пътя или файла не съществува или необходимата споделена библиотека не съществува.
ENOEXEC	Целта на пътя или файла е невалидна програма или е предназначена за различна архитектура.
ENOMEM	Няма достатъчно памет на ядрото за изпълнение на нова програма.
ENOTDIR	Непълнен компонент в пътя не е директория.
EPERM	Файловата система, на която се намира пътят или файла, е монтирана nosuid, потребителят не е root и пътя или файл има зададен suid или sgid бит.
ETXTBSY	Целта или файла е отворен за писане от друг процес.

Display File Source Code

- Initialize the variables
- Execute **nano show-source.c**
- If execution is unsuccessful, output a message and exit

show-source.c

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char * argv[])
{
    int ret = execl("/bin/nano", "nano", "show-source.c", NULL);
    if (ret == -1)
    {
        printf("Error running nano!\n");
        return EXIT_FAILURE; // -1
    }
    return EXIT_SUCCESS; // 0
}
```

Starting a Child Process

```
#include<sys/types.h>
#include<unistd.h>

pid_t fork(void);
pid_t getpid(void);
pid_t getppid(void);
```

- A successful call to **fork()** creates a new process, identical in almost all aspects to the calling process.
- The parent process receives the identifier (**pid**) of the child process, and the child process receives zero.
- On error and the child process is not created, **fork()** returns -1 and the error (**errno**) can be:

Error	Description
EAGAIN	The kernel failed to allocate certain resources, such as a new process identifier (pid).
ENOMEM	There is insufficient kernel memory to execute the request.

Program fragment for starting a child process:

fork.c

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>

int main(int argc, char * argv[])
{
    pid_t pid = fork();
    if(pid == -1)
    {
        printf("Error forking a process!\n");
        return EXIT_FAILURE; // -1
    }
    else if(pid == 0)
    {
        printf("Child process!\n");
    }
    else
    {
        printf("Parent process!\n");
    }
    return EXIT_SUCCESS; // 0
}
```

More information: [Linux Programming](#)

Print Process IDs

- Initialize the functions and variables.
- Create a child process and save its identifier (**pid**).
- If the value of **pid** is positive, then the parent process is running. Print the identifier of the parent process and **ppid**.
- If the value of **pid** is zero, then the child process is running. Print the identifier of the child process and **ppid**.
- If the value of **pid** is -1, then an error has occurred. Output an appropriate message and exit.

print-pids.c

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

void printPIDs(char* name)
{
    printf ("-----The %s process:\n", name);
    printf ("My pid=%d\n", getpid());
    printf ("Parent's pid=%d\n", getppid());
    printf ("-----\n");
}

int main(int argc, char* argv[])
{
    pid_t pid = fork();
    if(pid == -1)
    {
        printf("ERROR!\n");
        return EXIT_FAILURE;
    }
    else if(pid > 0)
    {
        printPIDs("PARENT");
    }
    else if (!pid) // pid==0
    {
        printPIDs("CHILD");
        exit(0);
    }
    return EXIT_SUCCESS;
}
```

Waiting for and Terminating a Child Process

```
#include<stdlib.h>
void exit(int status);
```

- Calling **exit()** performs some basic shutdown steps and then instructs the kernel to terminate the process.
- The status parameter (**status**) is used to indicate the exit status of the process.
- When a process finishes, the kernel sends a **SIGCHLD** signal to the parent.

```
pid_t pid = fork();
if (pid > 0) {
    printPIDs("PARENT");
    wait(&child_status);
}
else if (!pid) {
    printPIDs("CHILD");
    exit(0);
}
else if (pid == -1) {
    printf("ERROR");
    return EXIT_FAILURE;
}
```

Before terminating the process, the C library performs the following steps:

- Calls all functions registered with **atexit()** or **on_exit()**, in reverse order of their registration.
- Flushes all standard open input-output streams.
- Removes all temporary files created with the **tmpfile()** function.

These steps complete all the work that the process must do in user space, so **exit()** calls the system call **_exit()** to allow the kernel to handle the rest of the termination process:

```
#include <unistd.h>
void _exit (int status);
```

The process can be terminated if a signal is sent whose default action is to terminate the process. Such signals are **SIGTERM** and **SIGKILL**.

When a process finishes, the kernel sends a **SIGCHLD** signal to the parent process. By default, this signal is ignored and the parent takes no action. Processes can choose to handle this signal through the **signal()** or **sigaction()** system calls.


```
#include<sys/types.h>
#include<sys/wait.h>
pid_t wait(int status);
int WIFEXITED(status);
int WIFSIGNALED(status);
int WIFSTOPPED(status);
int WIFCONTINUED(status);
int WEXITSTATUS(status);
int WTERMSIG(status);
int WSTOPSIG(status);
int WCOREDUMP(status);
```

- **wait()** returns the value of the identifier (**pid**) of a child process or -1 on error.
- If no child process has terminated, the call blocks until a child process finishes.
- **WIFEXITED** returns true if the process terminated normally.
- On normal termination, **WEXITSTATUS** provides the eight bits that are passed to **_exit**.
- **WIFSIGNALED** returns true if a signal caused the process termination.
- In case of termination by signal, **WTERMSIG** returns the number of that signal.
- In case of termination by signal, **WCOREDUMP** returns true if the process dumped core in response to receiving the signal.
- **WIFSTOPPED** and **WIFCONTINUED** return true if the process was stopped or continued.
- If **WIFSTOPPED** is true, **WSTOPSIG** provides the number of the signal that stopped the process.

When an error occurs, there are two possible values:

Error	Information
ECHILD	The calling process has no child processes.
EINTR	The call terminated early due to a received signal.

Execute and Print Status

Check if we have enough arguments. Create a child process and save its identifier (pid). The parent process waits for the child process and exits on error. On success, the termination status is printed. The child process executes the command in the arguments.

exec-stat.c

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>

int main(int argc, char * argv[])
{
    int pid, status;
    if (argc < 2) {
        printf("Usage: %s command, [arg1 [arg2]...]\n", argv[0]);
        return EXIT_FAILURE;
    }
    printf("Starting %s...\n", argv[1]);
    pid = fork();
    if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror("execvp");
        return EXIT_FAILURE; // Never get there normally
    }
    else {
        if (wait(&status) == -1) {
            perror("wait");
            return EXIT_FAILURE;
        }
        if (WIFEXITED(status)) printf("Child terminated normally with exit code %i\n", WE
        if (WIFSIGNALED(status)) printf("Child was terminated by a signal #%i\n", WTERMSI
        if (WCOREDUMP(status)) printf("Child dumped core\n");
        if (WIFSTOPPED(status)) printf("Child was stopped by a signal #%i\n", WSTOPSIG(st
    }
    return EXIT_SUCCESS;
}
```

Zombies and Simple Signal Handling

When a child process dies before the parent process, the kernel places it in a special state called zombie (**zombie**). The process in this state waits for the parent process to query its status, and only then does the child process cease to exist as a zombie. If the parent process never queries the child process's status, then the zombie becomes a ghost (**ghost**), which is very bad practice. If the parent process terminates before its child processes, then their parent becomes the init process. The init process, in turn, periodically waits for all its child processes, thus ensuring that none of them remain zombies.

Waiting for a process:

```
#include<sys/wait.h>

pid_t waitpid(pid_t pid, int *wstatus, int options);
```

Signal handling:

```
#include<signal.h>

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void);
};
```

But what happens if the parent process dies before the child process, or if it dies before it has a chance to wait for its zombies? Every time a process terminates, the Linux kernel reviews the list of child processes and reparents them to the init process (process identifier pid = 1). This ensures that no process will be left without a parent. The init process, in turn, periodically waits for all its child processes, thus ensuring that none of them remain zombies for too long. Although this is still considered good practice, this safeguard means that short-lived processes should not worry excessively about waiting for all their children.

Zombie Game

Define a signal handler for terminating execution. Initialize variables for using the signal handler. Set action for **SIGCHLD**, exit on error. Do the following 10 times: create a child process and save its identifier (**pid**). The parent process prints the identifier of the child process. The child process prints a message and exits.

zombie-test.c

```
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>

static void sigchld_hdl (int sig)
{
    while (waitpid(-1, NULL, WNOHANG) > 0) {}
}

int main (int argc, char *argv[])
{
    struct sigaction act;
    memset (&act, 0, sizeof(act));
    act.sa_handler = sigchld_hdl;
    if (sigaction(SIGCHLD, &act, 0)) {
        perror ("sigaction");
        return 1;
    }
    for (int i = 0; i < 10; i++) {
        int pid = fork();
        if (pid == 0) {
            printf("I will leave no zombie\n");
            exit(0);
        } else printf("Created a process with the PID %i\n", pid);
    }
    while (1) sleep(1);
    return 0;
}
```

Process Exercise

Practice the materials from this topic by implementing the project presented below.

Temp File Generator

Write a program **temp-file-generator.c** that requires 1 argument: **files_count**. The program should generate **files_count** number of files named "**temp-file-NN.tmp**" (where the number *NN* is a number from 1 to *files_count*). Each file is created by a separate process and contains the identifier of the corresponding process as text 1000 times (*pid = 1001 should take 4 bytes per ID*). The parent process should be notified after each file is created.

Temp File Generator Plus

Write a new program **temp-file-generator-plus.c** by improving the previous program so that the processes write only to one file, where the sequences of process identifiers do not overlap (*we have 1000 times `pid_file_1`, then 1000 times `pid_file_2` ... then 1000 times `pid_files_count`*).

Chapter 5 – Inter-Process Communication

- [Inter-Process Communication Methods](#)
- [Signals](#)
- [Most Important Signals](#)
- [Signal Management](#)
- [Signal Handling](#)
- [Waiting for Signals](#)
- [Examining Signals](#)
- [Displaying Signal Information](#)
- [Reentrancy](#)
- [Anonymous and Named Pipes](#)
- [Working with Pipes](#)
- [Using Anonymous Pipes](#)
- [Using Named Pipes](#)
- [Inter-Process Communication Exercise](#)

Inter-Process Communication Methods

Structure	Information
File	A record saved on disk or a record synthesized on request from a file server, accessible by multiple processes.
Signal	A system message sent from one process to another, typically not used for data transfer but instead used for remote process control.
Socket	Data is sent over a network interface either to a different process on the same computer or to another computer on the network. Stream-oriented (TCP, data written through a socket requiring formatting to preserve message boundaries) or less commonly message-oriented (UDP, SCTP).
Unix domain socket	Similar to an internet socket, but all communication occurs within the kernel. Domain sockets use the file system as an address space. Processes refer to the domain socket as an inode and multiple processes can communicate with a single socket.
Message queue	A data stream similar to a socket, but which typically preserves message boundaries. Usually implemented by the operating system and allows multiple processes to read and write to the message queue without being directly connected to each other.
Pipe	A unidirectional data channel. Data written to the write end of the pipe is buffered by the operating system until read from the read end of the pipe. Bidirectional data streams between processes can be achieved by creating two pipes using standard input and output.
Named pipe	A pipe implemented through a file in the file system, instead of standard input and output. Multiple processes can read and write to the file as a buffer for IPC data.
Shared memory	Multiple processes are given access to the same block of memory, which creates a shared buffer so that processes can communicate with each other.

More information: [Inter Process Communication](#)

Signals

Signals are a mechanism for unidirectional asynchronous notifications. A signal can be sent from the kernel to a process, from a process to another process, or from a process to itself. Signals typically alert a process to some event, such as a segmentation fault or a user pressing the **Ctrl-C** key combination.

The Linux kernel implements around 30 signals (the exact number depends on the architecture).

Each signal is represented by a numeric constant and a text name. For example, **SIGHUP**, used to signal that a terminal hangup has occurred, has a value of 1 on the i386 architecture.

With the exception of **SIGKILL** (which always terminates the process) and **SIGSTOP** (which always stops the process), processes can control what happens when they receive a signal.

They can accept the default action, which can be terminating the process, terminating the process and dumping core, stopping the process, or nothing depending on the signal. Alternatively, processes can explicitly choose to ignore or handle signals. Ignored signals are silent.

Handled signals cause execution of a user signal handler function. The program jumps to this function as soon as the signal is received and (when the signal handler returns), program control resumes at the previously interrupted instruction.

Most Important Signals

signal	information	result
SIGHUP (1)	The process's controlling terminal is closed (most often the user has logged out).	Terminate
SIGINT (2)	The user uses the interrupt combination Ctrl-C.	Terminate
SIGABRT (6)	The abort() function sends this signal to the process that invokes it. The process then terminates and a core file is generated.	Terminate with core dump
SIGKILL (9)	This signal is sent by the kill() system call, it exists to provide a safe way to unconditionally kill a process.	Terminate
SIGSEGV (11)	This signal, whose name derives from segmentation violation, is sent to a process when it attempts invalid memory access.	Terminate with core dump
SIGTERM (15)	This signal is sent only via kill() and allows the user to gracefully terminate a process (default action).	Terminate
SIGCHLD (17)	When a process terminates or stops, the kernel sends this signal to the parent. The handler for this signal typically calls wait() to determine the child process identifier (pid) and exit code.	Ignored
SIGCONT (18)	The kernel sends this signal to a process when the process resumes after stopping (from SIGSTOP).	Ignored
SIGSTOP (19)	This signal is sent only via kill(), it unconditionally stops a process and cannot be caught or ignored.	Stop
SIGTSTP (20)	The kernel sends this signal to all processes in the foreground group when the user uses the Ctrl-Z combination.	Stop
SIGIO (29)	This signal is sent when an asynchronous I/O event is generated.	Terminate

signal	information	result
SIGUSR1(10), SIGUSR2(12)	These signals are available for user-defined purposes; the kernel never uses them. Processes can use SIGUSR1 and SIGUSR2.	Terminate

Signals in Linux

Execute following terminal command to view signal list:

```
kill -l
```

Result:

```

1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX

```

https://dsa.cs.tsinghua.edu.cn/oj/static/unix_signal.html

Execute following terminal command to view signal(7) manual:

```
man 7 signal
```

<https://man7.org/linux/man-pages/man7/signal.7.html>

Signal Management

method	description
sigemptyset()	initializes an empty signal set with all signals excluded from the set.
sigfillset()	initializes a full signal set including all signals.
sigaddset()	adds a signal to a set.
sigdelset()	removes a signal from a set.
sigprocmask()	retrieves and/or changes the signal mask of the calling thread.
sigaction()	changes the action of a signal.
sigwait()	waits for a signal.
strsignal()	returns a string describing the signal.

```
#include <signal.h>

int sigemptyset (sigset_t *set);
int sigaddset (sigset_t *set, int signo);
int sigprocmask (int how, const sigset_t *set, sigset_t *oldset);

int sigaction (int signo, const struct sigaction *act, struct sigaction *oldact);

struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void);
};
```

sigemptyset() initializes the signal set specified by set by marking it empty (all signals are excluded from the set).

sigaddset() adds a signal to the signal set specified by set, while **sigdelset()** removes the signal from the signal set specified by set. Both return 0 on success, or -1 on error, in which case the error is set to error code **EINVAL**, which means the signal is an invalid signal identifier.

The behavior of **sigprocmask()** depends on the value of one of the following:

Field	Description
SIG_SETMASK	Sets the signal mask of the calling process.
SIG_BLOCK	Signals are added to the signal mask of the calling process.
SIG_UNBLOCK	Signals are removed from the signal mask of the calling process.

The **sigaction** structure:

Field	Description
SA_HANDLER	Address of the handler SIG_IGN or SIG_DFL
SA_MASK	Signals to block
SA_FLAGS	Additional flags, such as SA_RESETHAND
SA_RESETHAND	Allows "one-shot" mode. The behavior of the given signal returns to default after the signal handler returns.

Calling **sigaction()** changes the behavior of the signal identified by **signo**, which can be any value except those associated with **SIGKILL** and **SIGSTOP**. If **act** is not NULL, the system call changes the current signal behavior as specified in **act**. If **oldact** is not NULL, the system call saves the previous behavior of the given signal (or the current one if **act** is NULL).

Signal Handling

- Define the signal handler function for **SIGTERM**
- Initialize a signal with only **SIGHUP** included and block signal handling
- Set an event handler for **SIGTERM**
- Print the process identifier (**pid**)
- Enter an infinite loop

signal-handler.c

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal Handler
void signal_handler(int i)
{
    printf ("Terminating\n");
    exit(EXIT_SUCCESS); // 1
}

// Main Program
int main(int argc, char ** argv)
{
    // Signal Structure Initialization
    struct sigaction sa;
    sigset_t newset;
    sigemptyset(&newset);
    sa.sa_flags = 0;
    sa.sa_handler = signal_handler;
    sigaction(SIGINT, &sa, NULL); // Ctrl+C

    // Print
    printf("My pid is %i\n", getpid());
    printf("Waiting...\n");

    // Forever
    while(1) sleep(1);

    return EXIT_FAILURE; // -1
}
```

Waiting for Signals

```
#include<signal.h>

int sigwait(const sigset_t *set, int *sig);
```

The **sigwait()** function suspends execution of the calling thread, waiting for one of the signals to become pending. The function accepts the signal (removes it from the pending signal list) and returns the signal number in **sig**. On success, **sigwait()** returns 0, on error it returns a positive number, the error number.

Signal Wait Example

Define the signal handler function for **SIGTERM**. Initialize a signal with only **SIGHUP** included and block signal handling. Set an event handler for **SIGTERM**. Print the process identifier (**pid**). Wait for an incoming **SIGHUP** signal and output an appropriate message. If an error occurs, terminate the program.

signal-wait.c

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Main program
int main(int argc, char ** argv)
{
    // Signal Structure Initialization
    int sig;
    struct sigaction sa;
    sigset_t newset;
    sigemptyset(&newset);
    sigaddset(&newset, SIGINT); // Ctr + C
    sigprocmask(SIG_BLOCK, &newset, 0);
    sigaction(SIGTERM, &sa, 0);

    // Print
    printf("My pid is %i\n", getpid());
    printf("Waiting...\n");

    // Forever
    while(!sigwait(&newset, &sig))
    {
        printf("SIGINT recieved\n");
        return EXIT_SUCCESS; // 0
    }

    return EXIT_FAILURE; // -1
}
```

Examining Signals

```
#include<string.h>
#include<signal.h>

char *strsignal(int sig);

extern const char * const sys_siglist[];

int sigfillset(sigset_t *set);

int sigdelset(sigset_t *set, int signum);
```

Calling **strsignal()** returns a pointer to a description of the signal given by **signo**.

sys_siglist is an array of strings containing the names of the signals supported by the system, indexed by signal number.

Displaying Signal Information

- Define the signal handler function for **SIGTERM**
- Initialize a signal set with only **SIGTERM** excluded
- Set an event handler for **SIGTERM**
- Print the process identifier (**pid**)
- Wait for and output appropriate messages for incoming signals
- If an error occurs, terminate the program

signal-info.c

```
#include<stdio.h>
#include<stdlib.h>
#include<signal.h>
#include<string.h>
#include<unistd.h>

void signal_handler(int sig)
{
    printf("Signal %i: %s\n", sig, strsignal(sig));
    exit(EXIT_SUCCESS); // 0
}

int main(int argc, char* argv[])
{
    int sig;
    struct sigaction sa;
    sigset_t sset;
    sigfillset(&sset);
    sigdelset(&sset, SIGTERM);
    sigprocmask(SIG_SETMASK, &sset, 0);
    sa.sa_handler = signal_handler;
    sigaction(SIGTERM, &sa, 0);

    printf("My pid is %i\n", getpid());

    while(!sigwait(&sset, &sig))
    {
        printf("Signal %i: %s\n", sig, strsignal(sig));
    }

    return EXIT_FAILURE;
}
```

Reentrancy

When the kernel sends a signal, the process is usually executing some code. This can happen in the middle of an important operation that, if interrupted, will place the process in an inconsistent state. For example, the data structure the process is working with is only half updated or the computation is only partially complete. The process may even be handling another signal.

The signal handler (**signal handler**) does not know exactly what code is being executed by the process at the moment the signal appears, this can happen at any time. For this reason, it is very important that every signal handler (**signal handler**) you write is very careful about the actions it performs and the data it works with.

Signal handlers (**signals handlers**) must take care not to make assumptions about what the process is doing when it is interrupted. In particular, they must be cautious when modifying global and shared data. In general, good practice is for the signal handler to never work with global data.

What about system calls or other library functions? What will happen if the process is in the middle of writing to a file or allocating memory, and a signal handler writes to the same file or calls **malloc()**? What will happen if, when a signal appears, the process is in the middle of calling a function that uses a static buffer, such as **strsignal()**?

For all these reasons, it is very important that every signal handler (**signal handler**) in the process is a **reentrant** function.

reentrant is a function that is safe to call from itself (or simultaneously from another thread in the same process). To qualify as **reentrant**, the function must not manipulate static data, must only manipulate data provided by the stack or data provided to it by the calling function, and must not execute a function that is not **reentrant**.

Standard C functions that are safe to use:

accept, access, aio_error, aio_return, aio_suspend, alarm, bind, cfgetispeed, cfgetospeed, cfsetispeed, cfsetospeed, chdir, chmod, chown, clock_gettime, close, connect, creat, dup, dup2, execl, execve, _Exit & _exit, fchmod, fchown, fcntl, fdatasync, fork, fpathconf, fstat, fsync, ftruncate, getegid, geteuid, getgid, getgroups, getpeername, getpgrp, getpid, getppid, getsockname, getsockopt, getuid, kill, link, listen, lseek, lstat, mkdir, mkfifo, open, pathconf, pause, pipe, poll, posix_trace_event, pselect, raise, read, readlink, recv, recvfrom, recvmsg, rename, rmdir, select, sem_post, send, sendmsg, sendto, setgid, setpgid, setsid, setsockopt, setuid, shutdown, sigaction, sigaddset, sigdelset, sigemptyset, sigfillset, sigismember, signal, sigpause, sigpending, sigprocmask, sigqueue, sigset, sigsuspend, sleep, socket, socketpair, stat, symlink, sysconf, tcdrain, tcflow, tcflush, tcgetattr, tcgetpgrp, tcsendbreak, tcsetattr, tcsetpgrp, time, timer_getoverrun, timer_gettime, timer_settime, times, umask, uname, unlink, utime, wait, waitpid, write.

Anonymous and Named Pipes

Anonymous Pipes

An anonymous pipe (*English term: Anonymous pipe*) is a simple **FIFO** communication channel that can be used for unidirectional inter-process communication (IPC). Typically, the parent program opens anonymous pipes and creates a new process that inherits the other ends of the pipes. The anonymous pipe lasts only as long as the process lasts.

Named Pipes

A named pipe (*English term: Named pipe*) is an extension of the concept of traditional anonymous pipes. A named pipe can be identified by name and shown as a file in the system. A named pipe can continue to exist as long as the computer system is on, not just during the lifetime of the process. A named pipe can be deleted if it is no longer being used.

Working with Pipes

```
#include <unistd.h>

int pipe(int pipefd[2]);
```

pipe() - creates an anonymous pipe.

- The **pipefd** array is used to return two file descriptors pointing to the ends of the pipe.
- **pipefd[0]** refers to the read end of the pipe
- **pipefd[1]** refers to the write end of the pipe.

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```

mkfifo() - creates a named pipe (**FIFO**).

- **mode** specifies the permissions for the **FIFO** structure. It is modified by the **umask** method in the usual way: the permissions of the created file are (**mode & ~umask**).

Using Anonymous Pipes

- Open a pipe (**pipe**) and save the input/output file descriptors (**I/O FD**)
- Create a child process (**fork**)
- Parent process: close the input file descriptor (**input/read**)
 - Write a message to the output file descriptor (**output/write**)
 - Close the output file descriptor (**output/write**)
- Child process: close the output file descriptor (**output/write**)
 - Read a message from the input file descriptor (**input/read**)
 - Print the received message
 - Close the input file descriptor (**input/read**)

pipe-pass-message.c

```
#include<stdio.h>
#include<string.h>
#include<sys/types.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char ** argv)
{
    int pipefd[2];
    int pd = pipe(pipefd);
    if(pd == -1)
    {
        printf("Error pipe!\n");
        return EXIT_FAILURE;
    }

    int pid = fork();
    if(pid == -1)
    {
        printf("Error fork!\n");
        return EXIT_FAILURE;
    }

    if(pid > 0)
    {
        char* str = "Hello World!";
        printf("Parent pid %i\nSend: %s\n", getpid(), str);
        close(pipefd[0]);
        write(pipefd[1], (void*)str, strlen(str)+1);
        close(pipefd[1]);
    }
    else
    {
        printf("Child pid %i\nReceive: ", getpid());
        char buf[1024];
        int len;
        close(pipefd[1]);
        while((len = read(pipefd[0], buf, 1024)) != 0)
        {
            printf("%s", buf);
        }
        close(pipefd[0]);
        printf("\n");
    }
}
```

```
    return EXIT_SUCCESS;  
}
```


Using Named Pipes

The presented pair of programs implements a chat-based client/server application that demonstrates the use of named pipes.

pipe-chat-server.c

Create a file named `./pipe-chat-fifo`. Open `./pipe-chat-fifo` for writing, exit in case of error. Start a loop that runs until we receive "q" from the keyboard. Read a character (**key**) and write it to the file. If **key** is Enter, use **flush**. Close and delete the file.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>

#define FIFO_FILE "./pipe-chat-fifo"

int main(int argc, char ** argv)
{
    mkfifo(FIFO_FILE, 0600);
    FILE* f = fopen(FIFO_FILE, "w");
    if(f == NULL)
    {
        printf("Error open file!\n");
        return EXIT_FAILURE;
    }

    char key;
    do
    {
        key = getchar();
        fputc(key, f);
        if(key == 10) fflush(f);
    }
    while(key != 'q');

    fclose(f);
    unlink(FIFO_FILE);
    return EXIT_SUCCESS;
}
```

pipe-chat-client.c

Open a file named **./pipe-chat-fifo** for reading. Start a loop that runs until we receive **"q"** from the keyboard. Read a character (**key**). Output the received character to the console. Close and delete the file.

```
#include<stdlib.h>
#include<stdio.h>
#include<unistd.h>

#define FIFO_FILE "./pipe-chat-fifo"

int main(int argc, char ** argv)
{
    FILE* f = fopen(FIFO_FILE, "r");
    if(f == NULL)
    {
        printf("Error open file!\n");
        return EXIT_FAILURE;
    }

    char key;
    do
    {
        key = fgetc(f);
        putchar(key);
    }
    while(key != 'q');

    fclose(f);
    unlink(FIFO_FILE);
    return EXIT_SUCCESS;
}
```

Inter-Process Communication Exercise

Practice the materials from this topic by implementing the projects presented below.

Reverse Encryptor Decryptor

Let's define "Reverse Encryption" (**RE**) as a technique for protecting text messages in which the text of the messages is reversed before they are sent further. Write a program **reverse-encryptor-decryptor.c** that has two processes communicating via an anonymous pipe (**anonymous pipe**).

The following rules must be valid for both processes:

- **Parent process** must read text from the keyboard, then apply the RE technique and send the encoded message to the **Child process**.
- **Child process** must wait for incoming messages, print the encoded messages to the screen, then decode them and store them in the file **reverse-encryptor-decryptor.log**.

Encrypted Chat

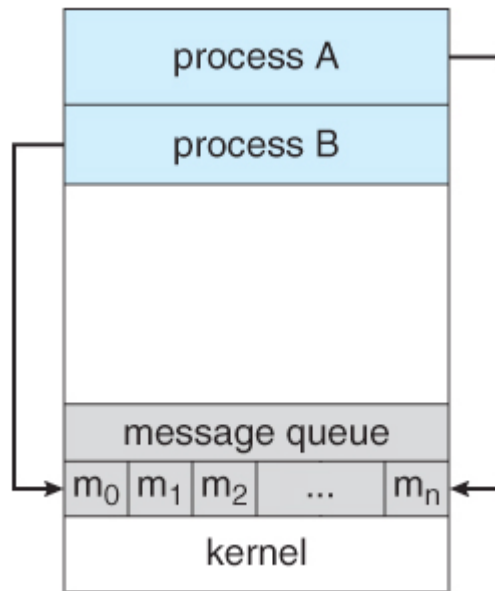
Write a pair of programs for server (**encrypted-chat-server.c**) and client (**encrypted-chat-client.c**). The server should mimic the behavior of the **Parent process** from the previous task, and the client should mimic the **Child process** respectively. The programs should use a named pipe (**named pipe**) as a communication method.

Chapter 6 - Inter-Process Communication Synchronization

- [Message Queue](#)
- [Libraries for Working with Messages](#)
- [Defining Common Data](#)
- [Message Server](#)
- [Message Client](#)
- [Shared Memory](#)
- [Libraries for Working with Shared Memory](#)
- [Defining Common Data](#)
- [Memory Server](#)
- [Memory Client](#)
- [Semaphore](#)
- [Libraries for Working with Semaphores](#)
- [Defining Common Data](#)
- [Semaphore Server](#)
- [Semaphore Client](#)
- [Inter-Process Communication Synchronization Exercise](#)

Message Queue

Message queues can be described as an internal linked list in the kernel's address space. Messages can be sent to the queue and retrieved from the queue in several different ways. Each message queue is uniquely identified by an IPC identifier.



More information: [Message Queues](#)

Libraries for Working with Messages

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg>

int msgget(key_t key, int msgflg);
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

The **msgget()** system call returns the System V message queue identifier associated with the value of the **key** argument.

The **msgsnd()** and **msgrcv()** system calls are used, respectively, to send and receive messages from a System V message queue. The calling process must have write permission on the message queue to send a message and read permission to receive a message.

msgctl() with **cmd = IPC_RMID** removes the message queue.

Defining Common Data

- Define **KEY** for the unique identification number of the queue.
- Define **MAXLEN** for the maximum message length.
- Define **msg_1_t** for the client message.
- Define **msg_2_t** for the response from the server.

message-types.h

```
#ifndef MSG_TYPES
#define MSG_TYPES

#define KEY 1274
#define MAXLEN 512

struct msg_1_t
{
    long mtype;
    int snd_pid;
    char body[MAXLEN];
};

struct msg_2_t
{
    long mtype;
    int snd_pid;
    int rcv_pid;
    char body[MAXLEN];
};

#endif
```

Message Server

- Create a message queue with the given **KEY**.
- Wait for a message of type **msg_1_t** from the queue.
- Print the data for the received messages.
- Prepare and send a response of type **msg_2_t**.
- Wait for **msg_1_t** confirmation.
- Remove the message queue.

message-server.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include "message-types.h"

int main(int argc, char ** argv)
{
    int msgid = msgget(KEY, 0777 | IPC_CREAT);
    if(msgid == -1)
    {
        printf("Error msgget.\n");
        return EXIT_FAILURE;
    }

    struct msg_1_t message1;
    msgrcv(msgid, &message1, sizeof(struct msg_1_t), 1, 0);
    printf("Client [pid = %i] send: %s\n", message1.snd_pid, message1.body);

    struct msg_2_t message2;
    message2.mtype = 2;
    message2.snd_pid = getpid();
    message2.rcv_pid = message1.snd_pid;
    strcpy(message2.body, "OK\n");
    msgsnd(msgid, &message2, sizeof(struct msg_2_t), 0);

    msgrcv(msgid, &message1, sizeof(struct msg_1_t), 1, 0);
    printf("Client [pid = %i] send: %s\n", message1.snd_pid, message1.body);

    msgctl(msgid, IPC_RMID, 0);
    return EXIT_SUCCESS;
}
```


Message Client

- Open a message queue with the given **KEY**.
- Output a message and exit the program in case of failure.
- Read the message body from the keyboard and set the other parameters of **msg_1_t**.
- Send **msg_1_t** message to the queue.
- Wait for **msg_2_t** and print it.
- Send **msg_1_t** confirmation.

message-client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <strings.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include "message-types.h"

int main(int argc, char ** argv)
{
    int msgid = msgget(KEY, 0666);
    if(msgid == -1)
    {
        printf("Error msgget.\n");
        return EXIT_FAILURE;
    }

    int i = 0;
    struct msg_1_t message1;
    while( (i < (MAXLEN - 1)) &&
           ((message1.body[i++] = getchar()) != '\n') );
    message1.body[i] = '\0';
    message1.mtype = 1;
    message1.snd_pid = getpid();
    msgsnd(msgid, &message1, sizeof(struct msg_1_t), 0);
    printf("Sended: %s\n", message1.body);

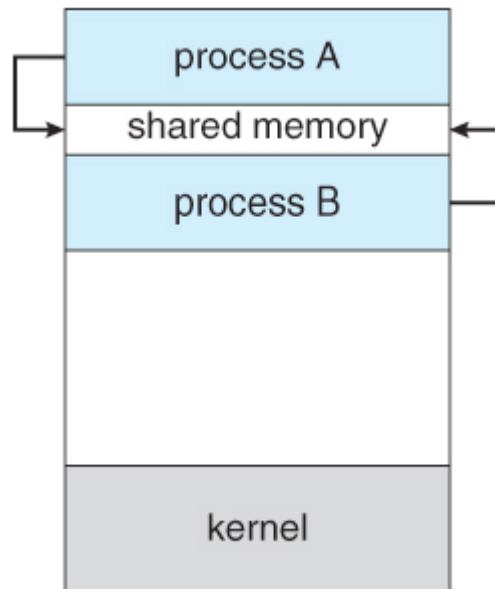
    struct msg_2_t message2;
    msgrcv(msgid, &message2, sizeof(struct msg_2_t), 2, 0);
    printf("Server [pid = %i] responded: %s\n", message2.snd_pid, message2.body);

    message1.mtype = 1;
    msgsnd(msgid, &message1, sizeof(struct msg_1_t), 0);

    return EXIT_SUCCESS;
}
```

Shared Memory

Shared memory can be described as mapping a region (segment) of memory that will be mapped and shared by more than one process. This is the fastest form of inter-process communication (**IPC**) because there is no intermediation - information is transferred directly from a memory segment into the address space of the calling process. The segment can be created by one process and then written to and read by any number of processes.



More information: [Shared Memory](#)

Libraries for Working with Shared Memory

```
#include<sys/types.h>
#include<sys/shm.h>
#include<sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);
int shmget(key_t key, size_t size, int shmflg);
void *shmat(int shmid, const void *shaddr, int shmflg);
int shmdt(const void *shmaddr);
int shmctl(int shmid, int cmd, struct shm_id *buf);
```

- **ftok()** = converts a pathname and a project identifier to a System V IPC key.
- **shmget()** = allocates a System V shared memory segment.
- **shmat()** = attaches the System V shared memory segment identified by **shmid** to the address space of the calling process.
- **shmdt()** = detaches the shared memory segment located at the address specified by **shmaddr** from the address space of the calling process.
- **shmctl()** with cmd = IPC_RMID - marks the segment to be destroyed.

Defining Common Data

Define the queue identification number **FTOK_FILE**. Define the shared memory structure **memory_block**:

- **server_lock** is 1 when the server is using the memory
- **client_lock** is 1 when the client is using the memory
- **turn** is 0 when waiting for a client message
- **turn** is 1 when waiting for a message from the server
- **read_last** is 0 when the client received the last message
- **read_last** is 1 when the server received the last message
- **string** holds the current message

shared-memory-types.h

```
#ifndef SHMEM_TYPES
#define SHMEM_TYPES

#define FTOK_FILE "./shared-memory-server"

#define MAX_LEN 512
#define FREE 1
#define BUSY 0
#define SERVER 1
#define CLIENT 0

struct memory_block
{
    int server_lock;
    int client_lock;
    int turn;
    int read_last;
    char string[MAX_LEN];
};

#endif
```

Memory Server

Create `./shared-memory-server`, exit on error. Allocate shared memory.

Configure the client's memory and write the message **"Hello!"** to it. While the current message is not "q". Lock the memory for the server, set the client's turn.

Wait if the client is using the memory. If the client has processed the last message

- indicate that the server has processed the current message, print the message from the client and return **"Ok!"** remove the server lock. Remove the shared memory.

shared-memory-server.c

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include "shared-memory-types.h"

int main(int argc, char ** argv)
{
    key_t key = ftok(FTOK_FILE, 1);
    if(key == -1)
    {
        printf("Error ftok.\n");
        return EXIT_FAILURE;
    }

    int shmid = shmget(key, sizeof(struct memory_block), 0666 | IPC_CREAT);
    if(shmid == -1)
    {
        printf("Error shmget.\n");
        return EXIT_FAILURE;
    }

    struct memory_block * mblock;
    mblock = (struct memory_block *) shmat(shmid, 0, 0);
    mblock->turn = CLIENT;
    mblock->server_lock = FREE;
    mblock->client_lock = FREE;
    mblock->read_last = SERVER;
    strcpy(mblock->string, "Hello");

    while(strcmp("q\n", mblock->string) != 0)
    {
        mblock->server_lock = BUSY;
        mblock->turn = CLIENT;
        while((mblock->client_lock == BUSY) &&
            (mblock->turn == CLIENT))
            if(mblock->read_last == CLIENT)
            {
                mblock->read_last = SERVER;
                printf("Sended: %s\n", mblock->string);
                if(strcmp("q\n", mblock->string) != 0)
                    strcpy(mblock->string, "OK");
                mblock->server_lock = FREE;
            }
    }
```

```
}

printf("Server exit.\n");
shmdt((void*)mblock);
shmctl(shmid, IPC_RMID, 0);
return EXIT_SUCCESS;
}
```


Memory Client

Create `./shared-memory-server`, exit on error. Allocate the shared memory. While the current message is not "q". Lock the memory for the client, set the server's turn. Wait if the server is using the memory. If the server has processed the last message - indicate that the client has processed the current message, print the message from the server, read a line from the keyboard and remove the client lock. Remove the shared memory.

shared-memory-client.c

```
#include<string.h>
#include<stdlib.h>
#include<stdio.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include "shared-memory-types.h"

int main(int argc, char ** argv)
{
    key_t key = ftok(FTOK_FILE, 1);
    if(key == -1)
    {
        printf("Error ftok.\n");
        return EXIT_FAILURE;
    }

    int shmid = shmget(key, sizeof(struct memory_block), 0666);
    if(shmid == -1)
    {
        printf("Error shmget.\n");
        return EXIT_FAILURE;
    }

    struct memory_block * mb;
    mb = (struct memory_block *) shmat(shmid, 0, 0);

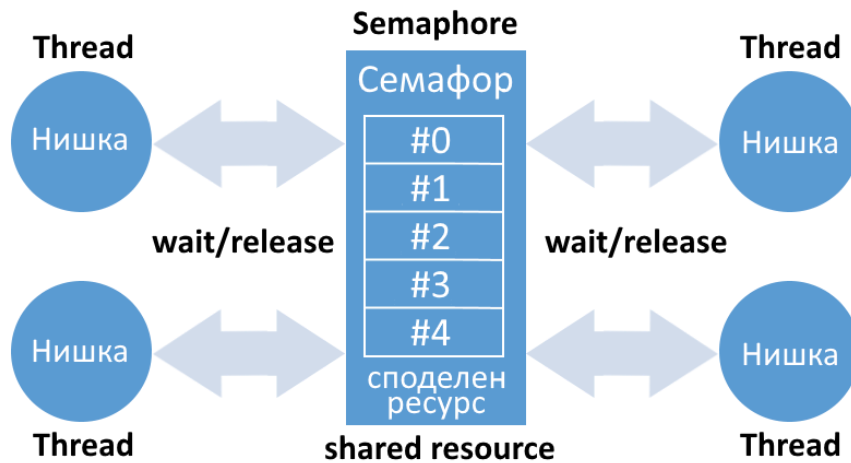
    while(strcmp("q\n", mb->string) != 0)
    {
        int i = 0;
        mb->client_lock = BUSY;
        mb->turn = SERVER;
        while((mb->server_lock == BUSY) &&
            (mb->turn == SERVER))
        {
            if(mb->read_last == SERVER)
            {
                mb->read_last = CLIENT;
                printf("Received: %s\n", mb->string);
                while( (i < (MAX_LEN - 1)) &&
                    ((mb->string[i++] = getchar()) != '\n') )
                {
                    mb->string[i] = '\0';
                    mb->client_lock = FREE;
                }
            }
        }
    }

    printf("Client exit.\n");
}
```

```
shmdt((void*)mb);  
return EXIT_SUCCESS;  
}
```

Semaphore

Semaphores are counters used to control access to shared resources by multiple processes (resource counters). They are used as a locking mechanism to prevent processes from accessing a specific resource while another process is performing operations on it. The resource counter decreases when a process starts using a resource and increases back when the resource is released. When the counter = 0, the resource is currently unavailable.



More information: [Semaphores](#)

Libraries for Working with Semaphores

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/sem.h>

int semget(key_t key, int nsems, int semflg);
int semctl(int semid, int semnum, int cmd, ...);
int semop(int semid, struct sembuf *sops, size_t nsops);
```

- **semget()** - returns the System V semaphore set identifier associated with the **key** argument.
- **semctl()** with **cmd = SETVAL** - Sets the value of **semval** to become **arg.val** for the **semnum**-th semaphore in the set.
- **semctl()** with **cmd = IPC_RMID** - removes the semaphore.
- **semop()** - performs operations on selected semaphores from the set designated by **semid**.

Defining Common Data

Define the lock file **semaphore-server**. Define the structure for the shared memory **memory_block**. The variable **string** contains the current message.

semaphore-types.h

```
#ifndef SEM_TYPES
#define SEM_TYPES

#define FTOK_FILE "./semaphore-server"

#define MAX_LEN 512

struct memory_block
{
    char string[MAX_LEN];
};

#endif
```

Semaphore Server

- Set up 2 semaphores - the first indicates that the server should read, the second for the client
- Create **semaphore-server**, exit on error
- Allocate and attach the shared memory and write "Hello!"
- Release the client's semaphore
- While the current message is not "q":
- Request the server semaphore resource
- If the current message is not "q", write "Ok!" to memory and release the client.
- Detach and remove the shared memory and semaphores

semaphore-server.c

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/sem.h>
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include"semaphore-types.h"

int main(int argc, char ** argv)
{
    key_t key = ftok(FTOK_FILE, 1);
    if (key == -1)
    {
        printf("Error ftok.\n");
        return EXIT_FAILURE;
    }

    int shmid = shmget(key, sizeof(struct memory_block), 0666 | IPC_CREAT);
    if(shmid == -1)
    {
        printf("Error shmget.\n");
        return EXIT_FAILURE;
    }

    int semid = semget(key, 3, 0666|IPC_CREAT);
    if(semid == -1)
    {
        printf("Error semget.\n");
        return EXIT_FAILURE;
    }

    struct sembuf buf[2];
    buf[0].sem_num = 0;
    buf[0].sem_flg = SEM_UNDO;
    buf[1].sem_num = 1;
    buf[1].sem_flg = SEM_UNDO;
    semctl(semid, 0, SETVAL, 0);

    struct memory_block * mb;
    mb = (struct memory_block *) shmat(shmid, 0, 0);
    strcpy(mb->string, "Hello!");
    buf[0].sem_op = -1;
    buf[1].sem_op = 1;
    semop(semid, (struct sembuf*) &buf[1], 1);
```



```

while (strcmp("q\n", mb->string) != 0)
{
    semop(semid, (struct sembuf*) &buf, 1);
    printf("Client: %s\n", mb->string);
    if (strcmp("q\n", mb->string) != 0)
        strcpy(mb->string, "Ok!");
    buf[0].sem_op = -1;
    buf[1].sem_op = 1;
    semop(semid, (struct sembuf*) &buf[1], 1);
}

printf("Server exit.\n");
shmdt((void *) mb);
shmctl(shmid, IPC_RMID, 0);
semctl(semid, 2, IPC_RMID);
return EXIT_SUCCESS;
}

```

Semaphore Client

- Set up 2 semaphores - the first indicates that the server should read, the second for the client
- Create **semaphore-server**, exit on error
- Attach the shared memory
- While the current message is not "q":
- Request the client semaphore resource and print the next message
- Read a new line from the keyboard into the shared memory
- Release the server's semaphore
- Detach the shared memory

semaphore-client.c

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "semaphore-types.h"

int main(int argc, char * argv[])
{
    key_t key = ftok(FTOK_FILE, 1);
    if (key == -1)
    {
        printf("Error ftok.\n");
        return EXIT_FAILURE;
    }

    int shmid = shmget(key, sizeof(struct memory_block), 0666);
    if (shmid == -1)
    {
        printf("Error shmget.\n");
        return EXIT_FAILURE;
    }

    int semid = semget(key, 2, 0666);
    if(semid == -1)
    {
        printf("Error semget.\n");
        return EXIT_FAILURE;
    }

    struct sembuf buf[2];
    buf[0].sem_num = 0;
    buf[0].sem_flg = SEM_UNDO;
    buf[1].sem_num = 1;
    buf[1].sem_flg = SEM_UNDO;
    buf[1].sem_op = -1;

    struct memory_block * mb;
    mb = (struct memory_block *) shmat(shmid, 0, 0);

    while (strcmp("q\n", mb->string) != 0)
    {
        semop(semid, (struct sembuf*) &buf[1], 1);
        printf("Server: %s\n", mb->string);
    }
}
```

```

    int i = 0;
    while ((i < (MAX_LEN - 1)) && ((mb->string[i++] = getchar()) != '\n' ));
    mb->string[i] = '\0';
    buf[0].sem_op = 1;
    buf[1].sem_op = -1;
    semop(semid, (struct sembuf*) &buf, 1);
}

printf("Client exits\n");
shmdt((void *) mb);
return EXIT_SUCCESS;
}

```

Inter-Process Communication

Synchronization Exercise

Practice the materials from this topic by implementing the projects presented below.

Double Shared Memory

Write a pair of server and client programs (**double-memory-server.c** and **double-memory-client.c**) that share memory blocks between them. The server should support up to 2 clients simultaneously, which should not conflict during operation. The server initializes the shared memory with greeting messages for its clients and waits for them to change with their messages. When the client writes a new message, the server responds with the same message but with the expression **Confirmed!** at the end.

The client is started with one argument: **client_number**, which identifies the client (1 or 2) to the server. The client should continuously print the message from the shared memory, read a line from the keyboard, and write it to the shared memory. The program should exit when the message "q" is entered.

Double Dumper

Write a program **double-dumper.c** that shows the changes in the shared memory from the **Double Shared Memory** project in real time. You may need to modify the server or client programs to support this functionality.

Chapter 7 – Sockets

- [Sockets in Linux](#)
- [Working with Sockets in C](#)
- [Using Unix Sockets](#)
- [File Socket Server](#)
- [File Socket Client](#)
- [Socket Pair](#)
- [Socket Pair Example](#)
- [Network Sockets](#)
- [Network Socket Server](#)
- [Network Socket Client](#)
- [Sockets Exercise](#)

Sockets in Linux

Most inter-process communications use the client-server model. One of the two processes, called the **client**, connects to the other process called the **server**, typically with a request for information. Each side of this bidirectional communication can be represented by a **socket**. The client and server processes create their own socket.

Socket Types

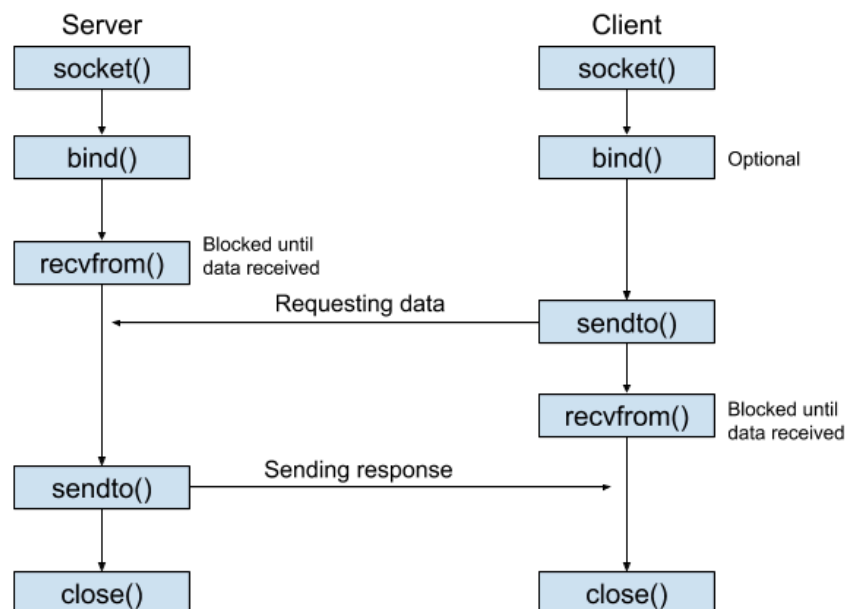
Unix domain socket is an endpoint for exchanging data between processes executing on the same host operating system.

Network socket is an endpoint for sending or receiving data within a host from a computer network.

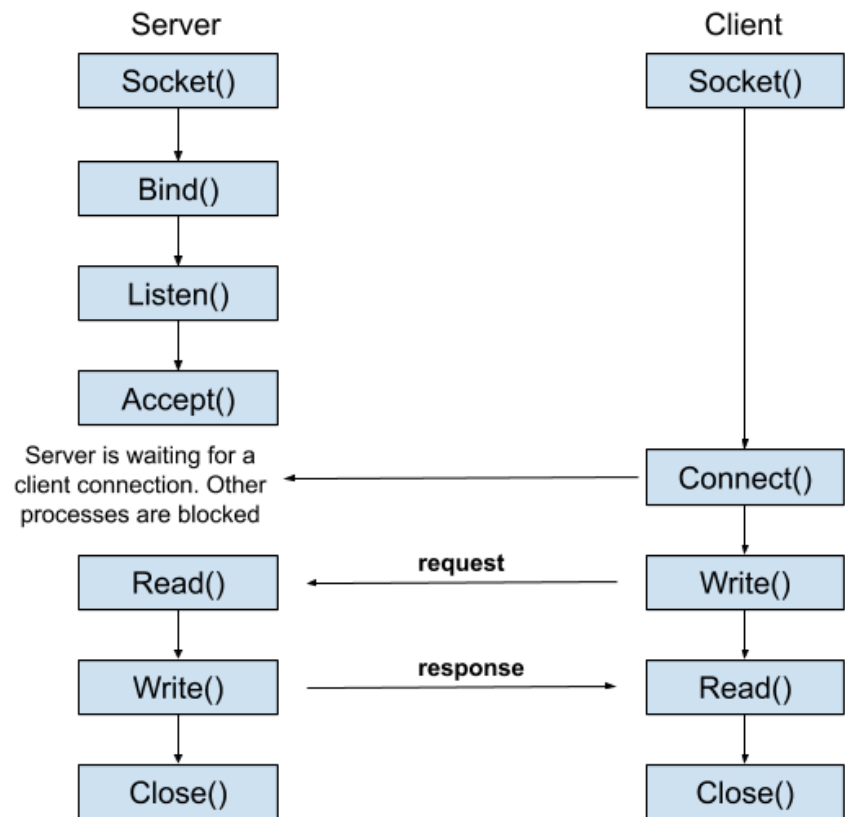
Network Socket Types

Sockets support the transmission of reliable **streams** of bytes, as well as ordered and unreliable transmission of **datagrams**. Stream sockets view communications as a continuous stream of characters, while Datagram sockets must read entire messages at once.

Communication via Datagram Socket



Communication via Stream Socket



Working with Sockets in C

method	description
socket ()	creates an endpoint for communication
bind ()	binds a name to a socket
recv (), recvfrom (), recvmsg ()	receives a message from a socket
send (), sendto (), sendmsg ()	sends a message to a socket
socketpair ()	creates a pair of connected sockets
listen ()	listens for connections on a socket
accept ()	accepts a connection on a socket
connect ()	initiates a connection on a socket
htons ()	converts unsigned short integer to network byte order
gethostbyname ()	returns a structure of type hostent for the given host name

Using Unix Sockets

```
#include<sys/types.h>
#include<sys/socket.h>

int socket(int domain, int type, int protocol);
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockad
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_ad
```

- **socket()** creates an endpoint for communication and returns a file descriptor that refers to that endpoint.
 - domain determines the protocol family which will be used: **AF_UNIX**, **AF_LOCAL**, **AF_INET**.
 - type determines the communication semantics: **SOCK_STREAM** or **SOCK_DGRAM**.
- When a socket is created with **socket()**, it exists in a name space (address space) but has no address assigned to it.
- **bind()** assigns the address specified in **addr** to the socket referred to by the file descriptor **sockfd** returned by the **socket()** method.
- **sockaddr** has the following structure:

```
struct sockaddr {
    sa_family_t sa_family;
    char sa_data[14];
}
```

File Socket Server

Open a Unix datagram socket, on error print a message and exit the program. Bind the socket to file **socket.file**, exit on error. Receive a message from the socket, exit on error. Print the message, close the socket and delete the socket file.

file-socket-server.c

```
#include<stdlib.h>
#include<unistd.h>
#include<stdio.h>
#include<string.h>
#include<errno.h>
#include<sys/types.h>
#include<sys/socket.h>

#define SOCK_NAME "socket.file"
#define BUF_SIZE 256

int main(int argc, char ** argv)
{
    int sock = socket(AF_UNIX, SOCK_DGRAM, 0);
    if(sock < 0)
    {
        printf("Error socket,\n");
        return EXIT_FAILURE;
    }

    struct sockaddr server, client;
    server.sa_family = AF_UNIX;
    strcpy(server.sa_data, SOCK_NAME);

    if(bind(sock, &server, strlen(server.sa_data) + sizeof(server.sa_family)) < 0)
    {
        printf("Error bind.\n");
        return EXIT_FAILURE;
    }

    char buf[BUF_SIZE];
    socklen_t len = sizeof(client);
    int bytes = recvfrom(sock, buf, sizeof(buf), 0, &client, &len);
    if(bytes < 0)
    {
        printf("Error recvfrom.\n");
        return EXIT_FAILURE;
    }
    buf[bytes] = '\0';
    client.sa_data[len] = 0;
    printf("Client: %s\n", buf);

    close(sock);
    unlink(SOCK_NAME);
}
```

```
    return EXIT_SUCCESS;  
}
```

File Socket Client

Open a Unix datagram socket, on error print a message and exit the program. Set the server socket name structure to **socket.file**. Send a message to the server and close the socket.

file-socket-client.c

```
#include<unistd.h>
#include<stdlib.h>
#include<stdio.h>
#include<string.h>
#include<errno.h>
#include<sys/types.h>
#include<sys/socket.h>

#define SOCK_NAME "socket.file"
#define BUF_SIZE 256

int main(int argc, char ** argv)
{
    int sock = socket(AF_UNIX, SOCK_DGRAM, 0);
    if(sock < 0)
    {
        printf("Error socket.\n");
        return EXIT_FAILURE;
    }

    struct sockaddr server;
    server.sa_family = AF_UNIX;
    strcpy(server.sa_data, SOCK_NAME);

    char buf[BUF_SIZE];
    strcpy(buf, "HELLO");
    sendto(sock, buf, strlen(buf), 0, &server, strlen(server.sa_data) + sizeof(server.sa_data));

    close(sock);
    return EXIT_SUCCESS;
}
```

Socket Pair

```
#include<sys/types.h>
#include<sys/socket.h>

int socketpair(int domain, int type, int protocol, int sv[2]);
```

The **socketpair()** call creates a pair of connected sockets in the specified domain, of the specified type, and using the optionally specified protocol.

The file descriptors used in referencing the new sockets are returned in **sv[0]** and **sv[1]** respectively.

On success, the method returns 0, on error -1, and error information is found in **errno**.

Socket Pair Example

Open a socket pair, on error print a message and exit the program.

Create a child process, on error print a message and exit the program.

Parent process: waits to receive a message by reading from the first socket, then sends a response with an appropriate reply message by writing to the first socket.

Child process: sends a message by writing to the second socket and waits to receive a reply message by reading from the second socket.

Finally, close the socket pair.

socket-pair.c

```
#include<unistd.h>
#include<stdlib.h>
#include<stdio.h>
#include<errno.h>
#include<sys/types.h>
#include<sys/socket.h>

#define MSG_HELLO "HELLO"
#define MSG_OK "OK"
#define BUF_SIZE 256

int main(int argc, char ** argv)
{
    char buf[BUF_SIZE];

    int sockets[2];
    if(socketpair(AF_UNIX, SOCK_STREAM, 0, sockets) < 0)
    {
        printf("Error socketpair.\n");
        return EXIT_FAILURE;
    }

    int pid = fork();
    if(pid < 0)
    {
        printf("Error fork.\n");
        return EXIT_FAILURE;
    }
    if(pid > 0)
    {
        {
            read(sockets[0], buf, sizeof(buf));
            write(sockets[0], MSG_OK, sizeof(MSG_OK));
            printf("Process (pid = %i) send message: %s\n", getpid(), buf);
        }
        if(pid == 0)
        {
            write(sockets[1], MSG_HELLO, sizeof(MSG_HELLO));
            read(sockets[1], buf, sizeof(buf));
            printf("Process (pid = %i) send message: %s\n", getpid(), buf);
        }

        close(sockets[0]);
        close(sockets[1]);
    }
}
```

```
    return EXIT_SUCCESS;  
}
```

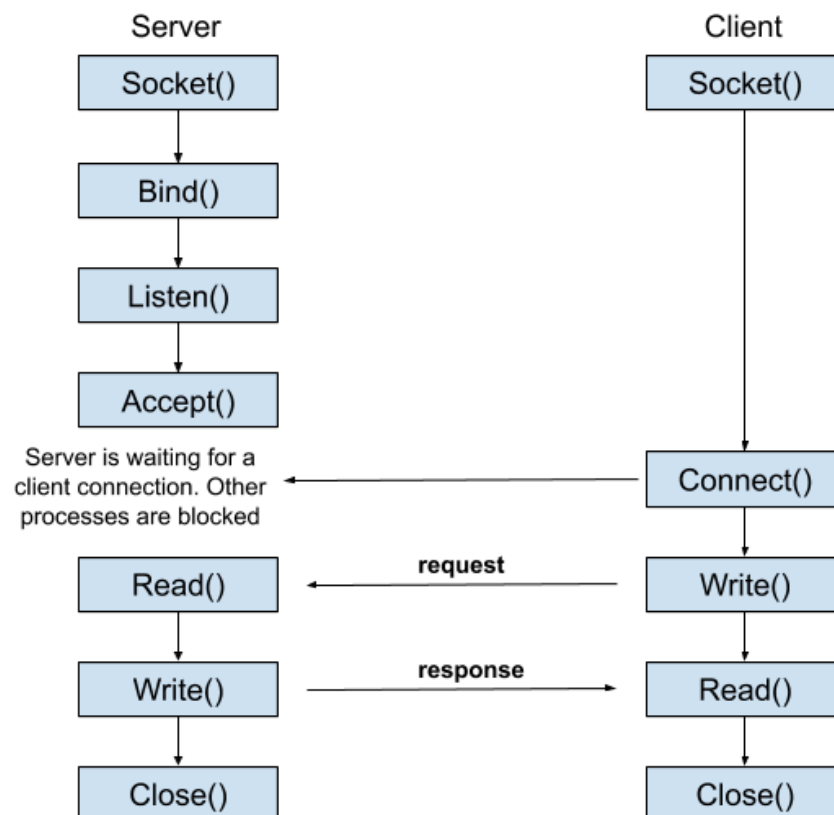
Network Sockets

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
extern int h_errno;

struct hostent *gethostbyname(const char *name);
uint16_t htons(uint16_t hostshort);
int listen(int sockfd, int backlog);
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

To accept connections, the following steps are performed:

1. A socket is created with **socket()**.
2. The socket is bound to a local address using **bind()**, so that other sockets can **connect()** to it.
3. The willingness to accept incoming connections and a queue limit for incoming connections are specified with **listen()**.
4. Connections are accepted with **accept()**.



Network Socket Server

Check input arguments, exit on error. Open an INET, STREAM socket, exit on error. Initialize the **sockaddr_in** structure for the server. Bind the socket to a port, exit on error. Start listening, allow only one connection. Accept a new connection, exit on error. Read from the socket into a buffer. Print the received message from the buffer. Send "OK" to the socket. Close the socket.

network-socket-server.c

```
#include <string.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <strings.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#define BUF_SIZE 256
int main(int argc, char ** argv)
{
    // Arguments Check!
    if (argc != 2)
    {
        printf("Usage: %s <port_number>\n", argv[0]);
        return EXIT_FAILURE; // return -1;
    }

    // Socket Create
    int sock_server = socket(AF_INET, SOCK_STREAM, 0);
    if (sock_server < 0)
    {
        printf("Socket Create Error: %d\n", errno);
        return EXIT_FAILURE; // return -1;
    }

    // Server Address
    struct sockaddr_in server_address;
    memset((char *) &server_address, 0, sizeof(server_address));
    server_address.sin_family = AF_INET;
    server_address.sin_addr.s_addr = htonl(INADDR_ANY);
    server_address.sin_port = htons(atoi(argv[1]));

    // Socket Bind
    if (bind(sock_server, (struct sockaddr *) &server_address, sizeof(server_address)) <
    {
        printf("Socket Bind Error: %d\n", errno);
        return EXIT_FAILURE; // return -1;
    }

    // Socket Listen
    if ((listen(sock_server, 5)) != 0)
    {
        printf("Socket Listen Error: %d\n", errno);
        return EXIT_FAILURE; // return -1;
    }
}
```

```

}

// Client Address
struct sockaddr_in client_address;
int len = sizeof(client_address);
int sock_client = accept(sock_server, (struct sockaddr *) &client_address, &len);
if (sock_client < 0)
{
    printf("Accept Socket Error: %d\n", errno);
    return EXIT_FAILURE; // return -1;
}

// Socket Read/Write
char buf[BUF_SIZE];
memset(buf, 0, BUF_SIZE);
read(sock_client, buf, BUF_SIZE-1);
buf[BUF_SIZE] = '\0';
printf("Received message: %s\n", buf);
write(sock_client, "OK", 2);

close(sock_client);
close(sock_server);
return EXIT_SUCCESS; // return 0;
}

```

Network Socket Client

Check input arguments, exit on error. Open an INET, STREAM socket, exit on error. Get the server host name, exit on error. Configure and connect the socket to the server address and port, exit on error. Display on screen that you are waiting for input and read a new text line from the keyboard. Send the entered text to the socket. Read a response from the socket and print it. Close the socket.

network-socket-client.c

```
#include<string.h>
#include<unistd.h>
#include<stdio.h>
#include<stdlib.h>
#include<errno.h>
#include<strings.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<netdb.h>
#define BUF_SIZE 256
int main(int argc, char ** argv)
{
    // Arguments Check!
    if (argc != 3)
    {
        printf("Usage: %s <ip_address> <port_number>\n", argv[0]);
        return EXIT_FAILURE; // return -1;
    }

    // Socket Create
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0)
    {
        printf("Socket Create Error: %d\n", errno);
        return EXIT_FAILURE; // return -1;
    }

    // Server IP Address
    struct in_addr addr;
    inet_aton(argv[1], &addr);
    struct hostent *server = gethostbyaddr(&addr, sizeof(addr), AF_INET);
    if (server == NULL)
    {
        printf("Host Not Found Error: %d\n", errno);
        return EXIT_FAILURE; // return -1;
    }

    // Server Address
    struct sockaddr_in serv_addr;
    memset((char *) &serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    strncpy((char *)&serv_addr.sin_addr.s_addr, (char *)server->h_addr, server->h_length);
    serv_addr.sin_port = htons(atoi(argv[2]));
```



```

// Socket Connect
if (connect(sock, (const struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
{
    printf("Socket Connect Error: %d", errno);
    return EXIT_FAILURE; // -1
}

// Socket Write/Read
printf("Write the message and press [Enter] to send:\n");
char buf[BUF_SIZE];
memset(buf, 0, BUF_SIZE);
fgets(buf, BUF_SIZE-1, stdin);
write(sock, buf, strlen(buf));
memset(buf, 0, BUF_SIZE);
read(sock, buf, BUF_SIZE-1);
printf("Received: %s\n", buf);

close(sock);
return EXIT_SUCCESS; // return 0;
}

```

Exercise

Practice the materials from this topic by implementing the project presented below.

Better Chat

Improve the pair of programs for network socket server **network-socket-server.c** and client **network-socket-client.c** by adding the following enhancements:

- The server should support multiple parallel client connections.
- Allow multiple messages per conversation until the client sends a **!quit** message to the server.

Chapter 8 – Threads

- [Introduction to Threads](#)
- [Thread Library](#)
- [Thread Management Functions](#)
- [Creating Threads](#)
- [Finishing Threads](#)
- [Joining Threads](#)
- [Thread Joining Example](#)
- [Thread Cancellation Functions](#)
- [Thread Cancellation Types](#)
- [Thread Cancellation Example](#)
- [Threads Exercise](#)

Introduction to Threads

Thread in computer science is short for thread of execution. Threads are a way for a program to split itself into two or more simultaneously or pseudo-simultaneously running tasks.

Threads and processes differ on different operating systems, but in general a thread is contained inside a process and different threads in the same process share the same resources, while different processes in the same multitasking operating system do not.

Threads consume fewer system resources compared to processes.

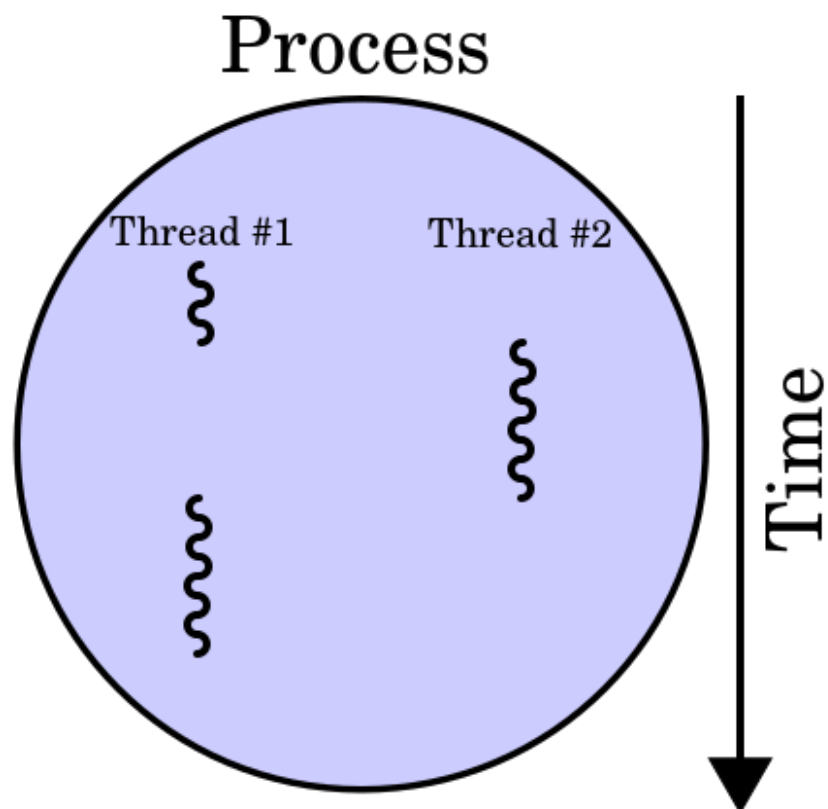


Image source: [Wikipedia](#)

Thread Library

The **pthread** library defines a set of C programming language types, functions and constants, which support applications with requirements for multiple flows of control, called threads, within a single process.

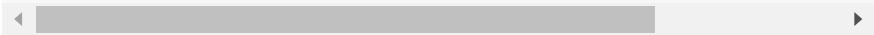
There are about 100 thread procedures, all with the prefix **pthread_** and they can be categorized into four groups:

Group	Description
Thread management	Thread management, creating and joining threads
Mutexes	Mutexes
Condition variables	Condition variables
Synchronization	Synchronization between threads using read/write locking

Note: In this topic we will focus only on "Thread management".

Thread Management Functions

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine
```



starts a new thread in the calling process

```
int pthread_join(pthread_t thread, void **retval);
```

waits for the specified thread to terminate

```
void pthread_exit(void *retval);
```

terminates the calling thread and returns a value via `retval`, which (if the thread is joinable) is available to another thread in the same process that calls **`pthread_join`**.

Creating Threads

Initially your **main()** program consists of a single, default thread. All other threads must be explicitly created by the programmer. `pthread_create` creates a new thread and makes it executable. This routine can be called any number of times from anywhere within your code.

pthread_create arguments:

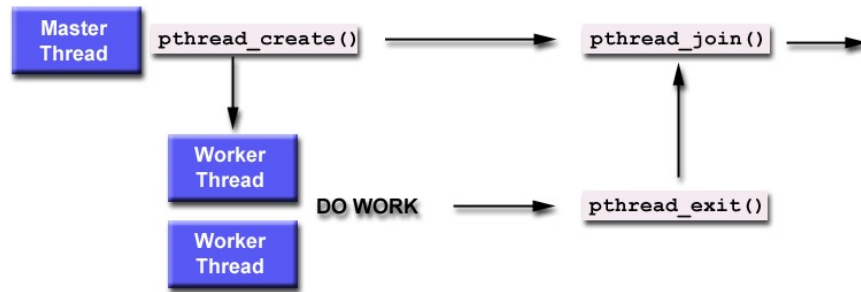
- **thread**: a unique identifier for the new thread, returned by the subroutine.
- **attr**: an attribute object that may be used to set thread attributes. You can specify a thread attributes object, or NULL for the default values.
- **start_routine**: the routine that the thread will execute once it is created.
- **arg**: a single argument that may be passed to `start_routine`. It must be passed by reference as a pointer cast of type void. NULL may be used if no argument is to be passed.

Finishing Threads

There are several ways in which a thread may be terminated:

- The thread returns normally and its work is done.
- The thread calls **pthread_exit** regardless of whether its work is done or not.
- The thread is cancelled by another thread via **pthread_cancel**.
- The entire process is terminated due to calling **exec()** or **exit()**
- If **main()** finishes first, without explicitly calling **pthread_exit**

Joining Threads



`pthread_join()` blocks the calling thread until the thread with identifier **`threadid`** terminates.

The programmer can obtain the termination status of the thread if indicated by a call to **`pthread_exit()`**.

Thread Joining Example

- Define a function for generating permutations
- Define a function for thread operation:
 - Allocate a vector with size **arg**
 - Initialize the vector with the first permutation and print it
 - While there is a next permutation: generate, print and synchronize
 - Clean up by freeing the vector memory and terminating the thread
- In the main function:
 - Start the thread function with size 4, exit on error
 - Start the thread function with size 3, exit on error
 - Wait for the first thread and print notification, exit on error
 - Wait for the second thread and print notification, exit on error
 - Print **Job Done!** and finish

threads-join.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>

// Print
void print(int *v, int n)
{
    for(int i=0;i<n;i++)
        printf("%i", v[i]);
        printf("\n");
}

// Swap
void swap(int *i, int *j)
{
    int temp = *i;
    *i = *j;
    *j = temp;
}

// Reverse
void reverse(int *v, int n)
{
    for(int i=0;i<(n/2); i++)
        swap(&v[i], &v[n-1-i]);
}

// Permute
int permute(int *v, int n)
{
    int i = n-1, j;
    while((i > 1) && (v[1] < v[i-1])) i--;
    if(v[i] > v[i-1])
    {
        j = n - 1;
        while(v[j] < v[i-1]) j--;
        swap(&v[j], &v[i-1]);
        reverse(&v[i], n-1);
        return 1;
    }
    return 0;
}

// Threads Function
void * func(void *arg)
{
    int i;
    int n = *(int*)arg; // n = 3 or 4
    int *v = malloc(sizeof(int)*n); // v[n]
    for(i=0;i<n;i++) v[i] = i+1; // 1,2 ... n
```

```

print(v,n); // 123 or 1234
while(permute(v,n) != 0)
{
    print(v,n);
    sync();
}
free(v);
pthread_exit(arg);
}

// Main
int main(int argc, char *argv)
{
    void *ret;
    int result;
    pthread_t thread1, thread2;

    // thread one
    int n1 = 4;
    result = pthread_create(&thread1, NULL, func, &n1);
    if(result != 0)
    {
        printf("Error Creating Thread One!\n");
        return EXIT_FAILURE; // return -1;
    }

    // tread two
    int n2 = 3;
    result = pthread_create(&thread2, NULL, func, &n2);
    if(result != 0)
    {
        printf("Error Creating Thread Two!\n");
        return EXIT_FAILURE; // return -1;
    }

    // thread one result
    result = pthread_join(thread1, &ret);
    if(result != 0)
    {
        printf("Error Joining Thread One!\n");
        return EXIT_FAILURE; // return -1;
    }
    printf("Thread one finished with result: %i\n", *(int*)ret);

    // thread two result
    result = pthread_join(thread2, &ret);
    if(result != 0)
    {
        printf("Error Joining Thread Two!\n");
    }
}

```

```
    return EXIT_FAILURE; // return -1;
}
printf("Thread two finished with result: %i\n", *(int*)ret);

// Job Done
printf("Job Done!\n");

return EXIT_SUCCESS; // return 0
}
```

Компилиране на програмата с изрична инструкция към компилатора да поддържа нишки:

```
gcc threads-join.c -o threads-join -lpthread
```

Thread Cancellation Functions

```
int pthread_cancel(pthread_t thread);
```

sends a cancellation request to the thread **thread**.

```
int pthread_setcancelstate(int state, int *oldstate);
```

sets the cancellation state of the thread via **state** to
PTHREAD_CANCEL_ENABLE or **PTHREAD_CANCEL_DISABLE**

```
int pthread_setcanceltype(int type, int *oldtype);
```

sets the cancellation type of the thread via **type** to
PTHREAD_CANCEL_DEFERRED or **PTHREAD_CANCEL_ASYNCHRONOUS**

Thread Cancellation Types

The `pthread_cancel()` function sends a cancellation request to a thread.

Whether and when the target thread reacts to the cancellation request depends on two attributes that are under the control of that thread: its cancelability state (enabled/disabled) and type:

- **Deferred** (default behavior) means that cancellation will be delayed until the thread calls a function `pthread_testcancel()`.
- **Asynchronous** means that the thread can be cancelled at any time (usually immediately, but the system does not guarantee this).

Thread Cancellation Example

- Create a thread
- Try to cancel it
- Wait for it to finish

```
pthread_create(&thread, NULL, thread_func, NULL);  
pthread_cancel(thread);  
pthread_join(thread, NULL);
```

- Mark the thread as non-cancellable
 - ... while it works
 - ... and until we allow cancellation
 - ... and reach a cancellation point

```
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);  
...  
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);  
pthread_testcancel();
```

- Make the thread cancellable at any time
- But for now mark it in a non-cancellable state
 - ... while it works
 - ... and until we allow cancellation
- The thread is cancelled automatically, no need to reach a cancellation point

```
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);  
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);  
...  
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
```


threads-cancel.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>

int i;
void * thread_func(void * arg)
{
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);
    for(i=0; i<4; i++)
    {
        sleep(1);
        printf("I am running verry important process %i ...\\n", i+1);
    }
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_testcancel();
    printf("YOU WILL NOT STOP ME!!!\\n");
}

int main(int argc, char * argv[])
{
    pthread_t thread;
    pthread_create(&thread, NULL, thread_func, NULL);
    while(i < 1) sleep(1);
    pthread_cancel(thread);
    printf("Requestd to cancel the thread!\\n");
    pthread_join(thread, NULL);
    printf("The thread is stopped!\\n");
    return EXIT_SUCCESS; // 0
}
```

Compiling the program with explicit instruction to the compiler to support threads:

```
gcc threads-cancel.c -o threads-cancel -lpthread
```

Threads Exercise

Practice the materials from this topic by implementing the project presented below.

ThreadedChat

Refactor the network server/client program pair from the previous topic with the following changes:

- Multi-user processing on the server should be done using **threads**.
- The server stops working when one of the clients sends a **!quitserver** message to the server.

Chapter 9 - Thread synchronization

- Thread exit
- Managing thread exit
- Thread exit example
- Why synchronization is necessary?
- Synchronization mechanisms
- Mutexes
- When to lock?
- Typical use of mutexes
- Race condition problems
- Thread-safe code
- Deadlock
- Creating and destroying mutexes
- Locking and unlocking mutexes
- Example: synchronization using a mutex
- Synchronizing with semaphores
- Example: synchronization using a semaphore
- Thread synchronization exercise

Thread exit

A thread can arrange for functions to be called when the thread exits. These functions are known as **thread cleanup handlers**. More than one cleanup handler can be registered for each thread. Handlers are pushed onto a **stack**, which means they are executed in the reverse order from which they were added.

When a thread is canceled or terminates by calling `pthread_exit()`, all registered cleanup handlers are popped and executed in the reverse order in which they were pushed onto the stack.

Managing thread exit

```
#include <pthread.h>

void pthread_cleanup_push(void (*routine)(void *), void *arg);

void pthread_cleanup_pop(int execute);
```

- The function `pthread_cleanup_push()` pushes a cleanup handler onto the calling thread's stack of cleanup handlers. When the handler is later invoked, the `arg` value will be passed to it.
- The function `pthread_cleanup_pop()` pops the handler at the top of the cleanup stack and optionally executes it if the `execute` argument is nonzero.

Thread exit example

- Define an exit function that frees the allocated memory
- Define the thread function: disable thread cancellation
- Allocate 1K of memory and print a message
- Add a thread cleanup handler, passing the allocated memory as a parameter
- Re-enable thread cancellation
- Print 5 messages over 5 seconds
- Call and execute the thread cleanup handler

thread-clean-exit.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#define MEM_SIZE 1024 // 1K
// Thread Cleanup Handler Function
void thread_cleanup_handler(void *arg)
{
    free(arg);
    printf("Freed %i bytes accolated memory.\n", MEM_SIZE);
}
// Thread Handler Function
void *thread_handler(void * arg)
{
    // Critical Section
    pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);

    void *mem = malloc(MEM_SIZE);
    printf("Allocated %i bytes memory.\n", MEM_SIZE);

    // Attach Thread Cleanup Hanler
    pthread_cleanup_push(thread_cleanup_handler, mem);

    // End of Critical Section
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);

    for(int i=0; i<5; i++)
    {
        sleep(1); // sleep for 1 second
        printf("Important process %i is running...\n", i+1);
    }

    // Execute Thread Cleanup Handler
    pthread_cleanup_pop(1);
}
// Main Function
int main(int argc, char *argv[])
{
    pthread_t thread;
    int result = pthread_create(&thread, NULL, thread_handler, NULL);
    if(result == -1)
    {
        printf("Error creating thread!\n");
        return EXIT_FAILURE; // -1
    }
    pthread_join(thread, NULL);
}
```

```
    return EXIT_SUCCESS; // 0  
}
```

Compile the program with an explicit instruction to link the pthreads library:

```
gcc thread-clean-exit.c -o thread-clean-exit -lpthread
```


Why synchronization is necessary

Thread synchronization is the coordination of concurrent execution between two or more threads that share critical resources.

Threads must be synchronized to avoid conflicts when accessing shared resources.

Otherwise, conflicts can arise when parallel threads attempt to modify a shared variable at the same time.

Synchronization mechanisms

The threads library provides three primary synchronization mechanisms:

- **Mutexes.** Mutual exclusion primitives that block access to variables from other threads. They enforce exclusive access by a single thread to a variable or a set of variables.
- **Joins.** A thread waits for other threads to finish.
- **Condition variables.** While mutexes provide synchronization by controlling thread access to data, condition variables allow threads to synchronize based on the actual value/state of the data.

Mutexes

Mutexes are used to prevent data inconsistencies that can occur when multiple threads operate on the same memory region at the same time, or to prevent **race conditions** when the order of memory operations matters.

A **race condition** often occurs when two or more threads perform operations on the same memory area, and the result depends on the order in which those operations are executed.

Mutexes are used to serialize access to shared resources such as memory. Whenever a global resource is accessed by more than one thread, the resource should have a mutex associated with it.

When to lock?

Thread 1	Thread 2	Balance
Read balance: \$1000		\$1000
	Read balance: \$1000	\$1000
	Deposit \$200	\$1000
Deposit \$200		\$1000
Update balance \$1000+\$200		\$1200
	Update balance \$1000+\$200	\$1200

In the example above a mutex should be used to protect the `balance` variable while a thread uses this shared data resource.

Thread 1	Thread 2	Balance
Read balance: \$1000		\$1000
Deposit \$200		\$1000
Update balance \$1000+\$200		\$1200
	Read balance: \$1200	\$1200
	Deposit \$200	\$1200
	Update balance \$1200+\$200	\$1400

In the example above thread 1 locks the `balance` resource and thread 2 is blocked until the mutex is released.

Typical use of mutexes

A typical sequence when using a mutex is:

- Create and initialize a mutex variable
- Multiple threads attempt to lock the mutex
- Only one succeeds and that thread owns the mutex
- The owning thread performs some set of actions
- The owner unlocks the mutex
- Another thread acquires the mutex and the process repeats
- Finally the mutex is destroyed

When multiple threads race for a mutex, the losing threads block on the call, but it is possible to use a `trylock()` variant where losing threads are not blocked.

When protecting shared data, every thread that needs to access the data must use the mutex. For example, if four threads update the same data but only one uses a mutex, the data can still become corrupted.

Race condition problems

Race conditions occur because, although the code may appear in the source in the order you expect it to run, threads are scheduled by the operating system and execute in an unpredictable order. It cannot be assumed that threads will run in the order they were created.

Threads may also run at different speeds. When threads execute concurrently (racing to finish), they can produce unexpected results.

Use **mutexes** and **joins** to achieve a predictable execution order and results.

Thread-safe code

`Thread-safe code` means that functions may be called by multiple threads safely.

This implies there are no static or global variables that other threads can corrupt or read concurrently. If static or global variables are used, either protect them with mutexes or rewrite the functions to avoid using shared state.

In C, local variables are allocated on the stack. Therefore any function that does not use static data or other shared resources is thread-safe.

Functions that are not thread-safe should only be used by one thread at a time in the program, and the program must ensure exclusive access.

Deadlock

Deadlock occurs when a mutex is not released (unlocked) and remains held, preventing progress.

This leads to the program stalling for an indefinite time. Deadlocks can be caused by poorly written application logic or incorrect use of joins.

Be careful when acquiring two or more mutexes in the same section of code. If the first `pthread_mutex_lock` succeeds but the second `pthread_mutex_lock` is blocked because another thread holds that mutex, the first mutex may end up preventing other threads from accessing data — including the thread that holds the second mutex.

Threads may wait indefinitely for a resource to be released, leading to a dead end. It is best to detect failures, release resources, and retry or abort before retrying.

Creating and destroying mutexes

```
#include <pthread.h>

void pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);

void pthread_mutex_destroy(pthread_mutex_t *mutex);

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Mutex variables must be declared with type `pthread_mutex_t` and initialized before they can be used. There are two ways to initialize a mutex variable:

- Statically, at declaration time using `PTHREAD_MUTEX_INITIALIZER` .
- Dynamically, using `pthread_mutex_init()` . This allows setting attributes for the mutex object.

Locking and unlocking mutexes

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- `pthread_mutex_lock()` is used by a thread to acquire the lock on the specified mutex variable. If the mutex is already locked by another thread, this call will block the caller until the mutex becomes available.
- `pthread_mutex_trylock()` attempts to lock the mutex but, if the mutex is already locked, returns immediately with an error code indicating the resource is busy. This can be useful to avoid deadlock situations.
- `pthread_mutex_unlock()` releases the mutex when called by the owning thread. This call must be made after the thread finishes using the protected data so other threads can acquire the mutex.

Example: synchronization using a mutex

- Define the `mutex` and `balance` variables
- Define the thread function that locks the mutex
- Add the deposit amount (`deposit`) to the balance (`balance`)
- Print the balance and unlock the mutex
- Create the first thread with a deposit of 200 and report an error on failure
- Create the second thread with a deposit of 200 and report an error on failure
- Wait for both threads to finish

thread-sync-mutex.c

```
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>

// Global Mutex
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;

// Global Variable Balance
int balance = 1000;

// Thread Handler Function to Deposit Money into the Balance
void *deposit(void *arg)
{
    // Critical Section
    pthread_mutex_lock(&mutex1);

    int deposit_amount = *(int *)arg;
    balance += deposit_amount;
    printf("Balance: %i\n", balance);

    pthread_mutex_unlock(&mutex1);
    // End of Critical Section
}

// Main Function
int main(int argc, char *argv[])
{
    int result;
    pthread_t thread1, thread2;
    int deposit_amount = 200;

    // Create Thread
    result = pthread_create(&thread1, NULL, &deposit, &deposit_amount);
    if(result == -1)
    {
        printf("Error creating thread one.\n");
        return EXIT_FAILURE; // -1
    }
    result = pthread_create(&thread2, NULL, &deposit, &deposit_amount);
    if(result == -1)
    {
        printf("Error creating thread two.\n");
        return EXIT_FAILURE; // -1
    }

    // Execute and join the results
```

```
pthread_join(thread1, NULL);  
pthread_join(thread2, NULL);  
  
return EXIT_SUCCESS; // 0  
}
```

Compile the program and link with pthreads:

```
gcc thread-sync-mutex.c -o thread-sync-mutex -lpthread
```

Synchronizing with semaphores

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);

int sem_wait(sem_t *sem);

int sem_post(sem_t *sem);

int sem_destroy(sem_t *sem);
```

- `sem_init()` initializes an unnamed semaphore at the address given by `sem` . The `value` argument specifies the initial value of the semaphore.
- `sem_wait()` decrements (locks) the semaphore pointed to by `sem` . If the semaphore value is zero, the call blocks until the semaphore becomes positive.
- `sem_post()` increments (unlocks) the semaphore pointed to by `sem` .
- `sem_destroy()` destroys the unnamed semaphore at the address given by `sem` .

Example: synchronization using a semaphore

- Declare a semaphore (`sem`)
- Declare the balance variable and initialize it to 1000
- Define the thread function that handles the argument and releases the semaphore
- Initialize the semaphore with value zero (meaning: busy)
- Start the first thread, passing the deposit amount (`deposit_amount`)
- Start the second thread, passing the deposit amount (`deposit_amount`)
- Wait for both threads to finish and destroy the semaphore

thread-sync-semaphore.c

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>

// Global Semaphore
sem_t sem;

// Global Variable Balance
int balance = 1000;

// Thread Handler Function to Deposit Money into the Balance
void *deposit(void *arg)
{
    // Critical Section
    sem_post(&sem);

    int deposit_amount = *(int *)arg;
    balance += deposit_amount;
    printf("Balance: %i\n", balance);

    sem_wait(&sem);
    // End of Critical Section
}

// Main Function
int main(int argc, char *argv[])
{
    int result;
    pthread_t thread1, thread2;
    int deposit_amount = 200;

    // Semaphore Initialization
    sem_init(&sem, 0, 0);

    // Create Thread
    result = pthread_create(&thread1, NULL, &deposit, &deposit_amount);
    if(result == -1)
    {
        printf("Error creating thread one.\n");
        return EXIT_FAILURE; // -1
    }
    result = pthread_create(&thread2, NULL, &deposit, &deposit_amount);
    if(result == -1)
    {
        printf("Error creating thread two.\n");
```



```
        return EXIT_FAILURE; // -1
    }

    // Execute and join the results
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    // Semaphore Destroy
    sem_destroy(&sem);

    return EXIT_SUCCESS; // 0
}
```

Compile the program and link with pthreads:

```
gcc thread-sync-semaphore.c -o thread-sync-semaphore -lpthread
```

Thread synchronization exercise

Practice the chapter material by implementing the project described below.

AdvancedChat

Modify the pair of network server/client programs from the previous chapter with the following changes:

- The server should accept an optional `-p` argument which, if present, enables a "polite mode". In polite mode the server should process client messages in rounds: when one client sends a message, that client's next message is not processed until all other connected clients have sent their messages. You may change the communication protocol between clients and server if needed to implement this behavior.
- The server should persist chat history to a file named `chat.log`.

Chapter 10 - Daemons

- [What are daemons?](#)
- [Daemon skeleton](#)
- [Chat daemon example](#)

What are daemons?

A **daemon** is a process that runs in the background and is not attached to a controlling terminal.

Typically, daemons are started at system boot, run as `root` or another dedicated user (for example `apache` or `postfix`), and perform system-level tasks.

Traditionally many daemon program names end with the letter `d` (for example `crond` and `sshd`), but this is a convention rather than a requirement.

Daemons commonly have two requirements:

- they should run as a child of `init` (or the system's init replacement),
- they must not be attached to a terminal.

Daemon skeleton

Steps typically required to daemonize a process:

1. Fork the parent process (`fork`) and let the parent exit; the child continues to run in the background.
2. Create a new session and become session leader (`setsid`). The calling process becomes the leader of a new session and process group and is disassociated from the controlling terminal (CTTY).
3. Catch and handle (or ignore) signals as needed.
4. Fork again (`fork`) and let the parent exit to ensure the daemon cannot reacquire a controlling terminal (only session leaders can acquire a controlling TTY).
5. Change the working directory (`chdir`) to a safe location (commonly `/`).
6. Set an appropriate file mode creation mask (`umask`).
7. Close all open file descriptors inherited from the parent (`close`).

daemonize.c

```
/*
 * daemonize.c
 * This example daemonizes a process, writes a few log messages,
 * sleeps 20 seconds and terminates afterwards.
 * This is an answer to the stackoverflow question:
 * https://stackoverflow.com/questions/17954432/creating-a-daemon-in-linux/17955149#17
 * Fork this code: https://github.com/pasce/daemon-skeleton-linux-c
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <syslog.h>

static void skeleton_daemon()
{
    pid_t pid;

    /* Fork off the parent process */
    pid = fork();

    /* An error occurred */
    if (pid < 0)
        exit(EXIT_FAILURE);

    /* Success: Let the parent terminate */
    if (pid > 0)
        exit(EXIT_SUCCESS);

    /* On success: The child process becomes session leader */
    if (setsid() < 0)
        exit(EXIT_FAILURE);

    /* Catch, ignore and handle signals */
    //TODO: Implement a working signal handler */
    signal(SIGCHLD, SIG_IGN);
    signal(SIGHUP, SIG_IGN);

    /* Fork off for the second time*/
    pid = fork();

    /* An error occurred */
    if (pid < 0)
```

```

        exit(EXIT_FAILURE);

/* Success: Let the parent terminate */
if (pid > 0)
    exit(EXIT_SUCCESS);

/* Set new file permissions */
umask(0);

/* Change the working directory to the root directory */
/* or another appropriate directory */
chdir("/");

/* Close all open file descriptors */
int x;
for (x = sysconf(_SC_OPEN_MAX); x>=0; x--)
{
    close (x);
}

/* Open the log file */
openlog ("firstdaemon", LOG_PID, LOG_DAEMON);
}

int main()
{
    skeleton_daemon();

    while (1)
    {
        //TODO: Insert daemon code here.
        syslog (LOG_NOTICE, "First daemon started.");
        sleep (20);
        break;
    }

    syslog (LOG_NOTICE, "First daemon terminated.");
    closelog();

    return EXIT_SUCCESS;
}

```

Sources

1. [How to Create a Daemon in C?](#)
2. [Basic skeleton of a linux daemon written in C](#)

Chat daemon

This example demonstrates a simple chat server that uses sockets and runs as a background daemon.

chat-daemon.c

```
// Headers
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <syslog.h>
#include <signal.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

// Common Constants
#define MAX_CLIENTS 100
#define BUFFER_SZ 2048
#define NAME_LEN 32

// Common Variables
static int client_count = 0;
static int uid = 10;
int option = 1;
int listenfd = 0, connfd = 0;
struct sockaddr_in serv;
struct sockaddr_in client;
pthread_t tid;
pthread_mutex_t clients_mutex = PTHREAD_MUTEX_INITIALIZER;

// Structure of Client
typedef struct {
    struct sockaddr_in address;
    int sockfd;
    int uid;
    char name[NAME_LEN];
} Client_t;

// Array of Clients
Client_t * clients[MAX_CLIENTS];

// Add Client
void add_client(Client_t * c1) {
    pthread_mutex_lock(&clients_mutex);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i] == NULL) {
```

```

        clients[i] = cl;
        break;
    }
}
pthread_mutex_unlock(&clients_mutex);
}

// Remove Client
void remove_client(int uid) {
    pthread_mutex_lock(&clients_mutex);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i] != NULL && clients[i]->uid == uid) {
            clients[i] = NULL;
            break;
        }
    }
    pthread_mutex_unlock(&clients_mutex);
}

// Send Message
void send_message(char * msg, int uid) {
    pthread_mutex_lock(&clients_mutex);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i] != NULL && clients[i]->uid != uid) {
            int wrt_status = write(clients[i]->sockfd, msg, strlen(msg));
            if (wrt_status < 0) break;
        }
    }
    pthread_mutex_unlock(&clients_mutex);
}

// Handle Client
void * handle_client(void * arg) {
    char buffer[BUFFER_SZ];
    char name[NAME_LEN];
    int leave_flag = 0;
    client_count++;

    Client_t * cli = (Client_t *) arg;

    // Joined
    sprintf(name, "%s:%i", inet_ntoa(cli->address.sin_addr), ntohs(cli->address.sin_port));
    strcpy(cli->name, name);
    sprintf(buffer, "%s joined!\n", cli->name); // Write to buffer
    send_message(buffer, cli->uid); // Print to all other clients
    bzero(buffer, BUFFER_SZ); // Clear buffer

    while (1) {

```

```

    if (leave_flag) break;

    int receive = recv(cli->sockfd, buffer, BUFFER_SZ, 0);
    if (receive > 0) {
        // Message
        char buff[BUFFER_SZ];

        // Attempts to overcome limitations: https://developers.redhat.com/blog/2019/08/
        strncpy(buff, cli->name, BUFFER_SZ - 1);
        buff[BUFFER_SZ - 1] = '\0';
        size_t n = strlen(buff);
        strncat(buff, "> ", BUFFER_SZ - n - 1);
        n = strlen(buff);
        strncat(buff, buffer, BUFFER_SZ - n - 1);

        // Print & Send
        send_message(buff, cli->uid);
    } else if (receive == 0 || strcmp(buffer, "exit") == 0) {
        // Left
        sprintf(buffer, "%s left!\n", cli->name);
        send_message(buffer, cli->uid);
        leave_flag = 1;
    } else {
        leave_flag = 1;
    }
    bzero(buffer, BUFFER_SZ);
}
close(cli->sockfd);
remove_client(cli->uid);
free(cli);
client_count--;
pthread_detach(pthread_self());

return NULL;
}

// Daemonize
static void daemonize() {
    // Fork off the parent process
    pid_t pid = fork();

    // An error occurred
    if (pid < 0) exit(EXIT_FAILURE);

    // Success: Let the parent terminate
    if (pid > 0) exit(EXIT_SUCCESS);

    // On success: The child process becomes session leader

```

```

if (setsid() < 0) exit(EXIT_FAILURE);

// Catch, ignore and handle signals
signal(SIGCHLD, SIG_IGN);
signal(SIGHUP, SIG_IGN);

// Fork off for the second time
pid = fork();

// An error occurred
if (pid < 0) exit(EXIT_FAILURE);

// Success: Let the parent terminate
if (pid > 0) exit(EXIT_SUCCESS);

// Set new file permissions
umask(0);
int stdiofd = open("/dev/null", O_RDWR);
dup(stdiofd);
dup(stdiofd);

// Open the log file
openlog("chat-daemon", LOG_PID, LOG_DAEMON);
}

// Main Method
int main(int argc, char * argv[]) {
    // Arguments check
    if (argc != 2) {
        printf("Syntax: ./chat-daemon [port]\n");
        return EXIT_FAILURE;
    }

    // Socket settings
    listenfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    serv.sin_family = AF_INET;
    serv.sin_addr.s_addr = INADDR_ANY;
    serv.sin_port = htons(atoi(argv[1]));

    // Bind
    if (bind(listenfd, (struct sockaddr *)&serv, sizeof(serv)) < 0) {
        printf("ERROR: bind()");
        return EXIT_FAILURE;
    }

    // Listen
    if (listen(listenfd, 10) < 0) {
        printf("ERROR: listen()");
    }
}

```

```

        return EXIT_FAILURE;
    }

    // Daemonize
    daemonize();

    // Working
    while (1) {
        socklen_t client_len = sizeof(client);
        connfd = accept(listenfd, (struct sockaddr *)&client, &client_len);

        // Check for max clients
        if (client_count + 1 == MAX_CLIENTS) {
            syslog(LOG_NOTICE, "Max clients connected! Connection rejected!\n");
            close(connfd);
            continue;
        }

        // Client settings
        Client_t * cli = (Client_t *) malloc(sizeof(Client_t));
        cli->address = client;
        cli->sockfd = connfd;
        cli->uid = uid++;

        // Add client to queue
        add_client(cli);
        pthread_create(&tid, NULL, &handle_client, (void *) cli);

        // Reduce CPU usage
        sleep(1);
    }

    closelog();
    return EXIT_SUCCESS;
}

```

Running the chat daemon

Server:

```

gcc chat-daemon.c -o chat-daemon -lpthread
./chat-daemon 5005

```

Clients:

nc 46.10.253.12 5005